Chaînes et processus de Markov

Files d'attente et application au Monopoly

Gaspard Nordin
30972
2022-2023

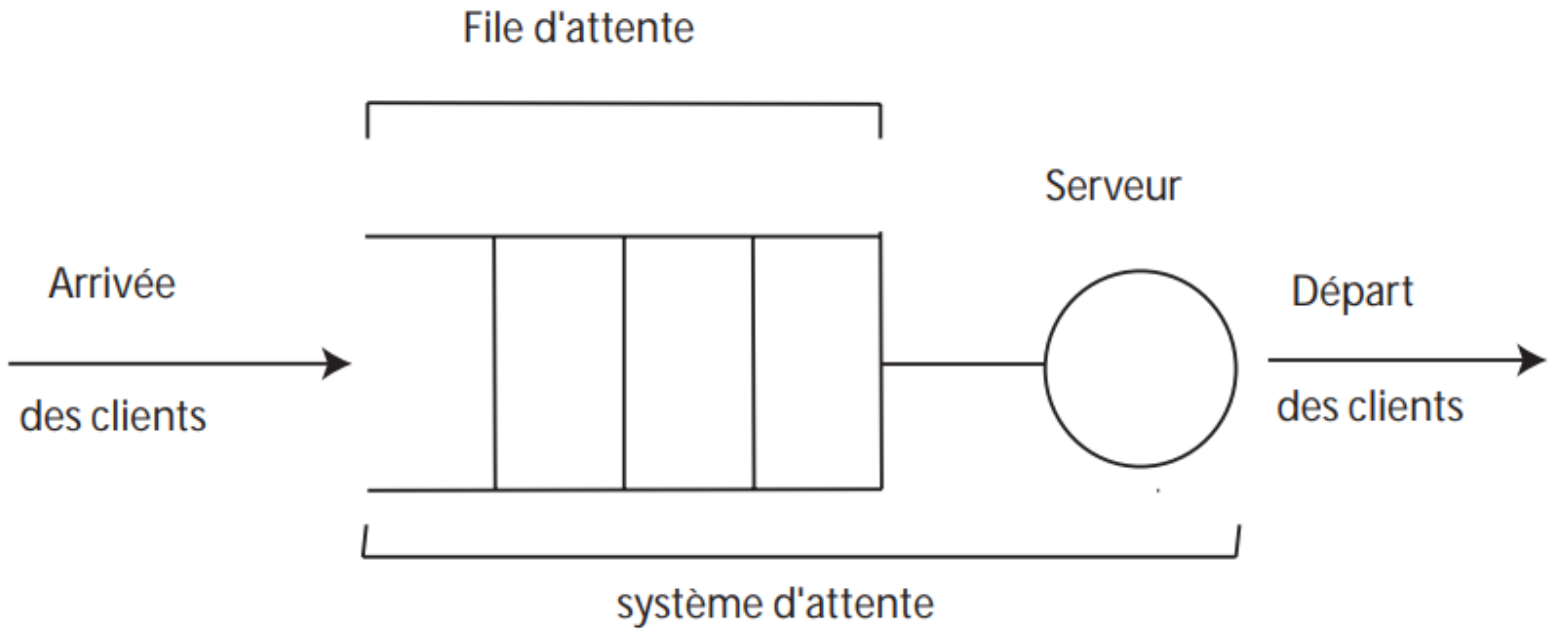
INTRODUCTION

- Phénomènes courants de la vie quotidienne.

- Présentes majoritairement dans les lieux peuplés



Notion de file d'attente



Plan de mon travail

- Modéliser une file d'attente simple M/M/1
- Comparer par simulation différents modèles de files d'attente
- Appliquer les chaînes de Markov à un cas simple : le Monopoly

1/File d'attente M/M/1

a) Caractéristiques de la file M/M/1
b) Processus de générateur infinitésimal
c) Obtention des paramètres opérationnels

2/Simulation de files d'attente

a) Validation de la M/M/1 à l'aide d'un modèle informatique
b) Une ou deux files d'attente M/M/1

3/Application : le Monopoly

A. Étude du plateau

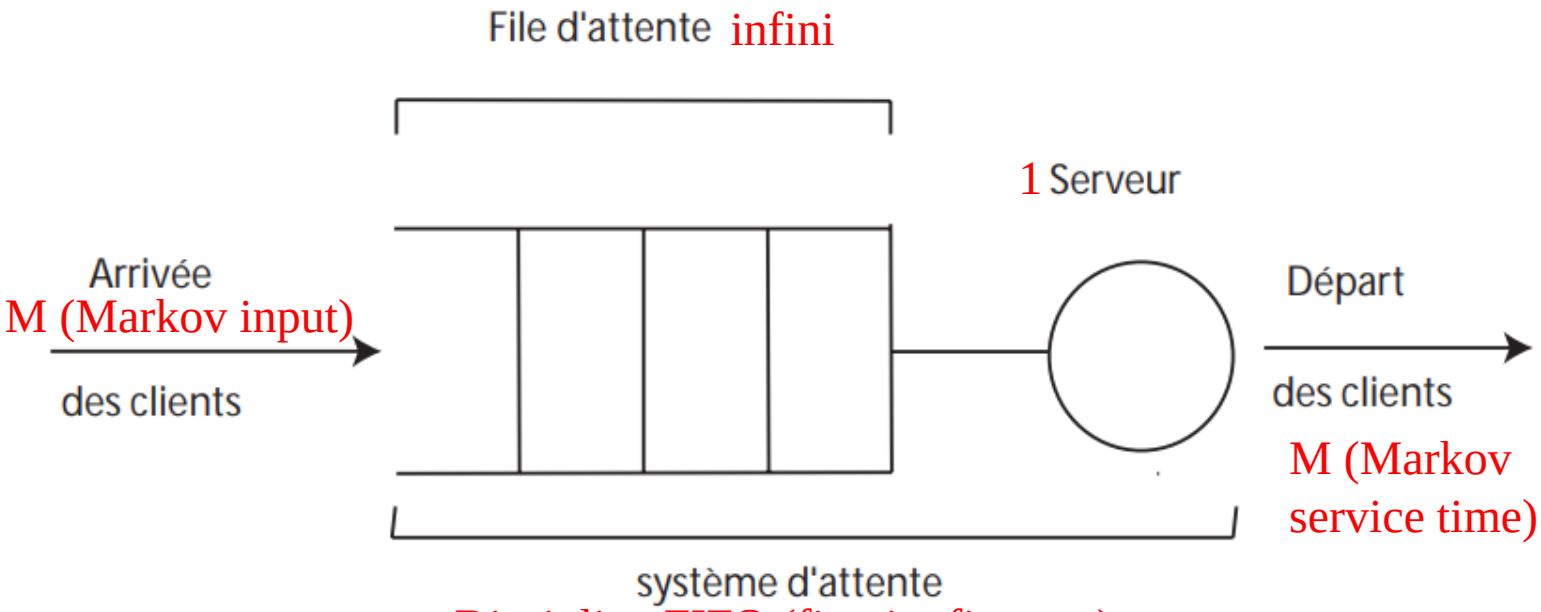
a) Modélisation par une chaîne de Markov
b) Obtention de la matrice de transition
c) Simulation informatique

B. Aspects financiers du jeu

a) Gain par propriété
b) Optimisation des dépenses
c) Temps de séjour court ou long ?

1/File d'attente M/M/1

a) Caractéristiques de la file M/M/1



PROCESSUS ALÉATOIRES

-Un espace de temps $\mathbb{R}_+$.

-Un espace des états $\mathbb{N}$.

-Une famille de variables aléatoires définies sur un même espace probabilisé et à valeurs dans $\mathbb{N}$: $(X_t)_{t\in\mathbb{R}_+}$

PROCESSUS DE MARKOV

A) Axiome de Markov

$$P([X_{t_{n+1}} = x_{n+1}] | \prod_{i=1}^{n} [X_{t_i} = x_i]) = P([X_{t_{n+1}} = x_{n+1}] | X_{t_n} = x_n)$$

B) Axiome d'homogénéité

$P([X_{t+s} = y] | [X_s = x])$ ne dépend que de t

Régime d'entrée : processus de Poisson

Un processus de comptage tel que $N_0 = 0$ est un processus de Poisson si :

1) $N_{t+s} - N_s \sim N_t$
2) Si $t_1 < t_2 < ... < t_n$, $N_{t_n} - N_{t_{n-1}}, ..., N_{t_1} - N_{t_0}$ sont indépendantes.
3) $\exists \lambda > 0, \forall t \geq 0, \forall k \in \mathbb{N}, \ N_t \sim \mathscr{P}(\lambda t)$

Régime de sortie : loi exponentielle

La loi exponentielle modélise la durée de vie d'un phénomène sans mémoire.

$$\exists \mu \in \mathbb{R}_+, \forall t \in \mathbb{R}_+, P(X > t) = e^{-\mu t}$$

$$\text{Densité de probabilité : } f(t) = \begin{cases} \mu e^{-\mu t} & \text{si } t \geqslant 0 \\ 0 & \text{si } t < 0 \end{cases}$$

b) Processus de générateur infinitésimal

$$P(t) = p_{i,j}(t) = P(X_t = j | X_0 = i)$$

$$\boxed{\overrightarrow{\Pi}(t) = (\Pi_i(t))_{i \in \mathbb{N}} \text{ avec } \Pi_i(t) = P(X_t = i)}$$

$$1)\ p_{i,j}(t) \geq 0 \text{ et } \sum_j p_{i,j}(t) = 1$$

$$2)\ P(s+t) = P(s)P(t)$$

$$3)\ \overrightarrow{\Pi}(s+t) = \overrightarrow{\Pi}(s)P(t)$$

Equation différentielle vérifié par P(t)

$$\text{Avec } P(t+h) = P(t)P(h)$$

$$\frac{P(t+h) - P(t)}{h} = \left(\frac{P(h) - P(0)}{h} \right) P(t)$$

$$A = \lim_{h \to 0} \left(\frac{P(h) - P(0)}{h} \right) = P'(0)$$

$$\text{donc } P'(t) = P(t)A \qquad \text{et P}(0) = I_d$$

$$\boxed{P(t) = e^{tA}}$$

Générateur de la file M/M/1

$$A = \begin{bmatrix} -\lambda & \lambda & 0 & \dots & 0 & \\ \mu & -(\lambda+\mu) & \lambda & \dots & 0 & \\ 0 & \ddots & \ddots & \ddots & \vdots & \\ \vdots & \ddots & \ddots & \ddots & 0 & \\ 0 & \dots & \dots & \mu & -(\lambda+\mu) & \lambda \\ 0 & \dots & \dots & 0 & \mu & -\mu \end{bmatrix}$$

λ : taux d'arrivée

μ : taux de service

Équation de Chapmann-Kolmogorov

$$P'(t) = AP(t) \text{ et } \overrightarrow{\Pi}(t) = \overrightarrow{\Pi}(0)P(t)$$

$$\overrightarrow{\Pi}'(t) = \overrightarrow{\Pi}(0)AP(t)$$

On se place en régime permanent $(\Pi_n = \lim_{t \to +\infty} \Pi_n(t))$:

$$\begin{cases} 0 = \lambda\Pi_0 - \mu_1\Pi_1 \\ \forall n \geq 1,\ 0 = \lambda\Pi_{n-1} - (\lambda + \mu)\Pi_n + \mu\Pi_n \\ \sum_{n=0}^{+\infty} \Pi_n = 1 \end{cases}$$

En posant $\rho = \dfrac{\lambda}{\mu}$:

$$\Pi_n = \Pi_0 \rho^n$$

$$\Pi_0 = \frac{1}{\sum_{n=0}^{+\infty} \rho^n}$$

c) Obtention des paramètres de performance

Π_n : probabilité que la file contienne n clients.

L : nombre moyen de clients dans la file d'attente.

$$L = \sum_{n=0}^{+\infty} n\Pi_n = \frac{\rho}{1-\rho}$$

W : temps moyen de séjour dans la file d'attente.

$$W = \frac{L}{\lambda} = \frac{1}{\mu(1-\rho)}$$

2) Simulations de files d'attente

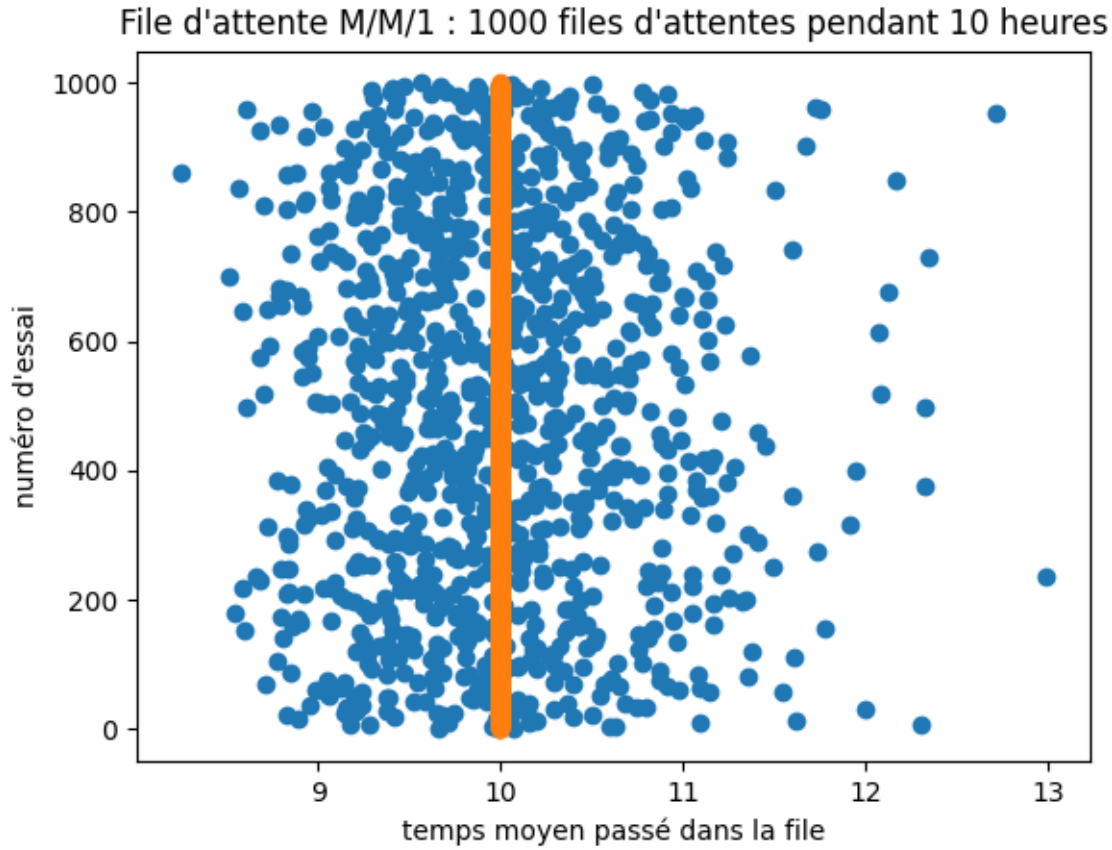
- Classe de clients : clients prioritaires, sexe
- Classe de serveurs : spécialisation, efficacité
- Traffic irrégulier : horaire de pointe/heure creuse

On introduit les données :

W : temps d'attente moyen

L : longueur moyenne de la file

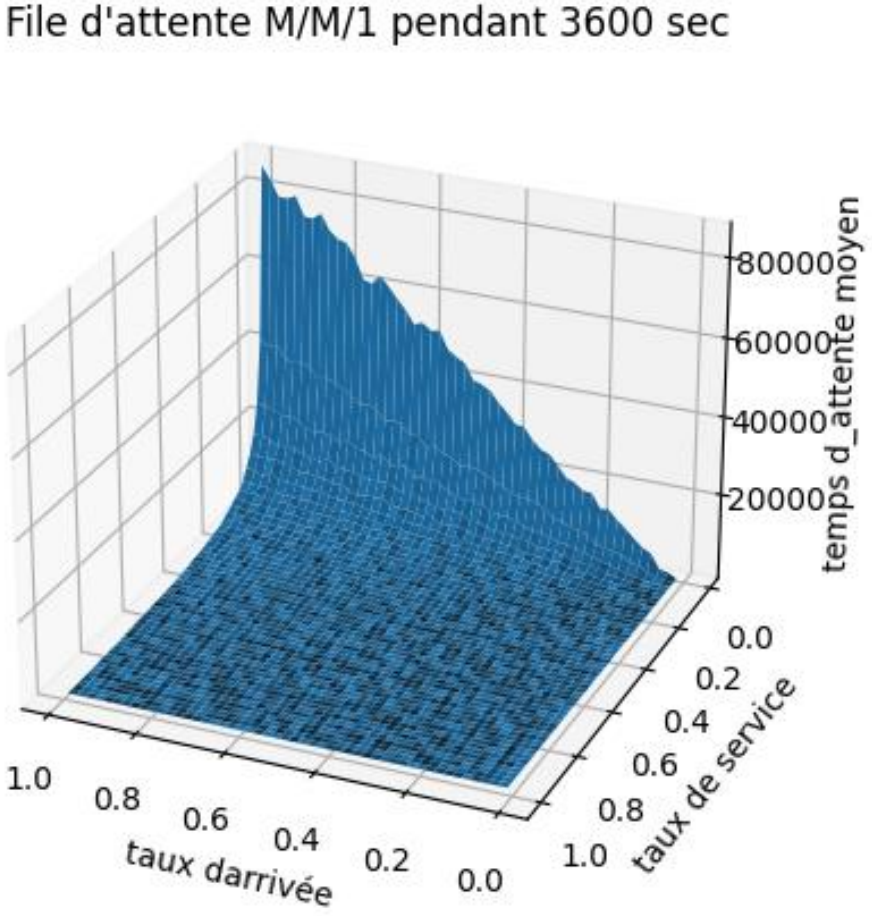
a) Validation de la file M/M/1



$$\lambda = 0.2 \qquad \mu = 0.3$$

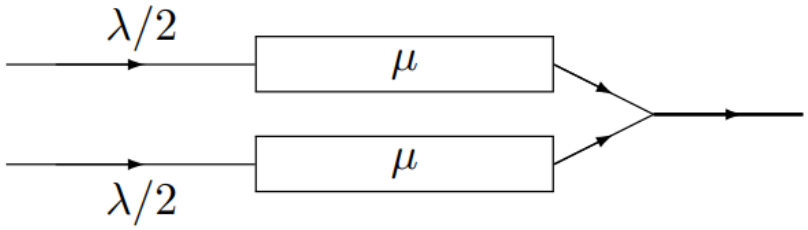
Valeur trouvée par la théorie : $W = 10\,\text{sec}$

Valeur trouvée par la simulation : $W = 9.99794339\,\text{sec}$

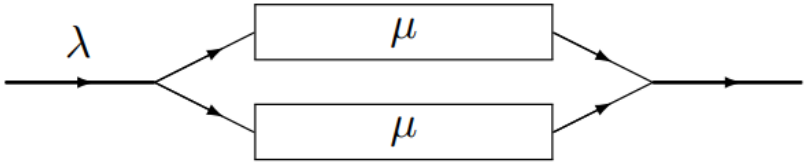


Évolution d'une file d'attente M/M/1
en fonction de ses paramètres d'entrée et de sortie

b) Une ou deux files d'attente M/M/1



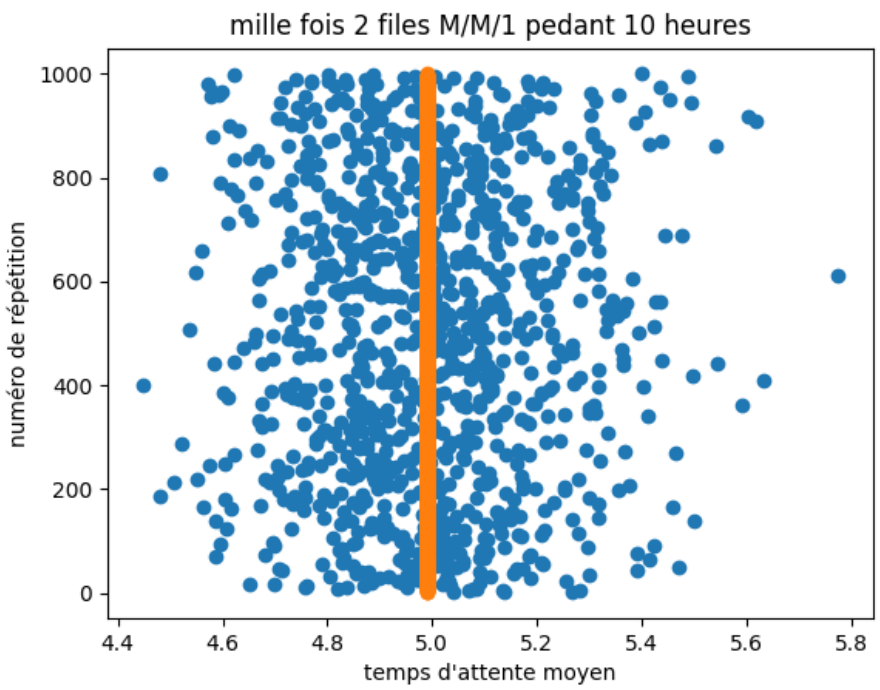
Deux files $M(\lambda/2)/M(\mu)/1$



Une file $M(\lambda)/M(\mu)/2$

	2 files $M\left(\frac{\lambda}{2}\right)/M(\mu)/1$	Comparaison	1 file $M(\lambda)/M(\mu)/2$
W	$\frac{2}{(2\mu-\lambda)}$	$>$	$\frac{4\mu}{4\mu^2-\lambda^2}$
L	$\frac{2\lambda}{2\mu-\lambda}$	$>$	$\frac{4\lambda\mu}{4\mu^2-\lambda^2}$

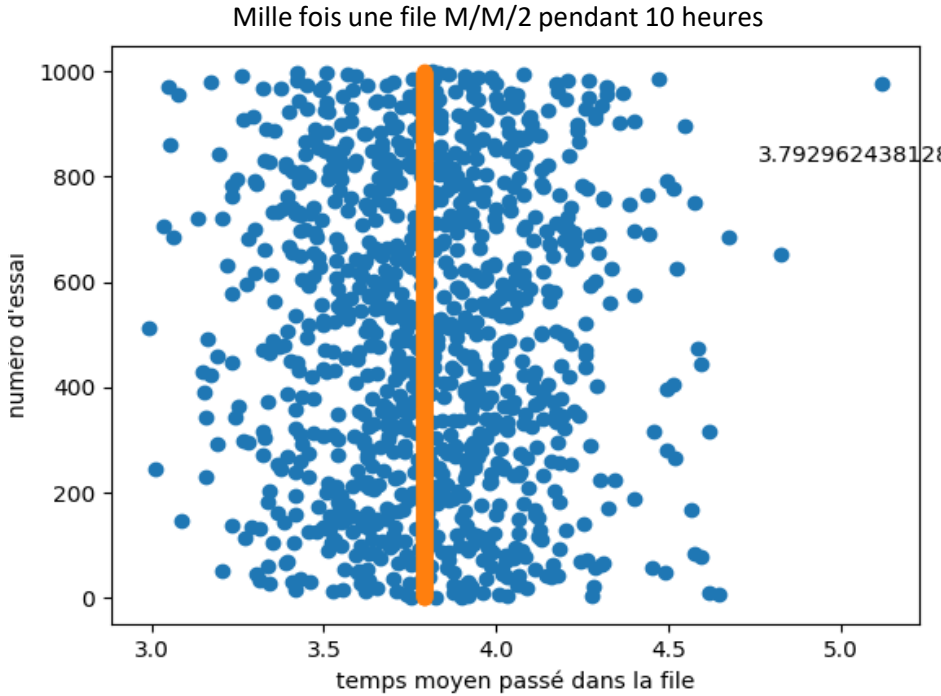
Réponse de l'algorithme à 1000 files d'attentes pendant 10 heures/Vérification du modèle théorique



$\lambda = 0.1 \qquad \mu = 0.3 \qquad$ pour les 2 files.

valeur théorique : $W = 5$

valeur expérimentale : $W = 4.99$



$\lambda = 0.2 \qquad \mu = 0.3$

Valeur théorique : $W = 3.8$

Valeur expérimentale : $W = 3.79$

Affiliation des clients par genre

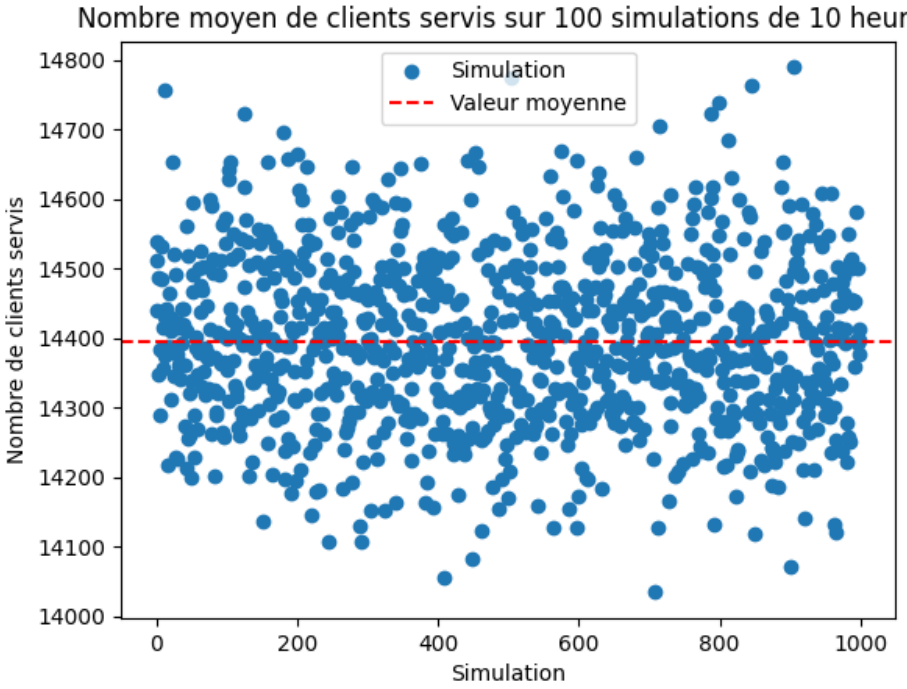
Les clients arrivent selon un taux d'arrivée : λ

On leur associe un genre selon une loi de Bernoulli.

A chaque genre on associe un serveur :

chaque client ne peut être servis que la par le serveur spécialisé dans son genre.

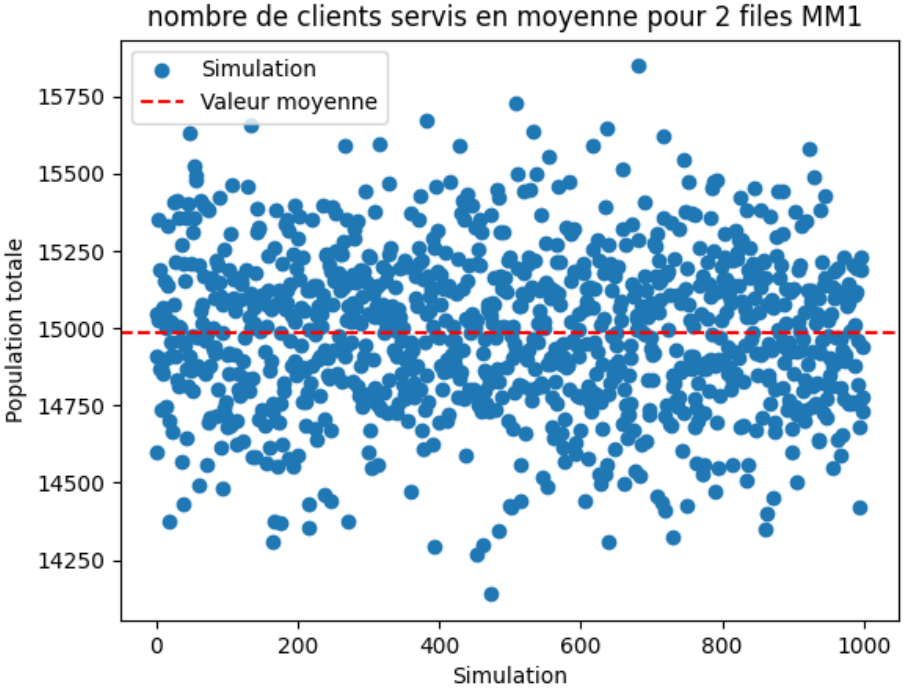
Comparaison des systèmes avec genre proportion homme/femme = 50/50



FILE D'ATTENTE M/M/2

avec clients selon le genre et serveur préférentiel

$\lambda = 0.4 \qquad \mu = 0.3$



Deux files d'attente M/M/1

spécialisée par genre.

$\lambda = 0.2 \qquad \mu = 0.3$

3) Application : le Monopoly

Un exemple simple de chaîne de Markov à temps discret



- Jeu à information complète
- Hasard : le lancer des dés
- Réflexion et prise de décision

Objectifs :

- Proposer une modélisation du Monopoly
- Établir des stratégies gagnantes

A. Étude du plateau

a) Modélisation par une chaîne de Markov

Même cadre que le processus de Markov de la partie 1, mais **à temps discret** :

- espace de temps : $\mathbb{N}$ (un lancer de dés par instant) ;
- espace des états : $E = [\![0\,;39]\!]$ (les 40 cases du plateau) ;
- suite $(X_n)_{n\in\mathbb{N}}$ à valeurs dans E, vérifiant les axiomes de Markov et d'homogénéité.

$$P(n) = \big(p_{i,j}(n)\big)_{i,j\in E} \text{ avec } p_{i,j}(n) = P(X_n = j \mid X_0 = i)$$

$$\text{Loi de } X_n : \ \overrightarrow{\Pi}(n) = \big(P(X_n = i)\big)_{i\in E}$$

Mêmes propriétés que $P(t)$ (démonstration en annexe) :

1) $p_{i,j}(n) \geq 0$ et $\sum_j p_{i,j}(n) = 1$ 2) $P(n+1) = P(n)P(1)$ 3) $\overrightarrow{\Pi}(n) = \overrightarrow{\Pi}(0)P(n)$

Le générateur A et la relation $P(t) = e^{tA}$ sont remplacés par :

$$\boxed{P(n) = P(1)^n \qquad \text{et} \qquad \overrightarrow{\Pi}(n) = \overrightarrow{\Pi}(0)\, P(1)^n}$$

Tout repose donc sur la matrice de transition $P(1)$, construite selon les règles du jeu.

b) Obtention de la matrice de transition

Première modélisation du Monopoly

Hypothèses :

1) Pas de cartes Chance et Caisse de communauté.
2) Omission de la règle des trois doubles consécutifs qui mène en prison.

Modélisation éloignée de la réalité.

Dé 2 \ Dé 1	1	2	3	4	5	6
1	2	3	4	5	6	7
2	3	4	5	6	7	8
3	4	5	6	7	8	9
4	5	6	7	8	9	10
5	6	7	8	9	10	11
6	7	8	9	10	11	12

$$P = \begin{bmatrix} 0 & 0 & \frac{1}{36} & \frac{2}{36} & \frac{3}{36} & \frac{4}{36} & \ddots & \ddots & 0 & 0 \\ 0 & 0 & 0 & \frac{1}{36} & \frac{2}{36} & \frac{3}{36} & \ddots & \ddots & 0 & 0 \\ 0 & 0 & 0 & 0 & \frac{1}{36} & \frac{2}{36} & \ddots & \ddots & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & \frac{1}{36} & \ddots & \ddots & 0 & 0 \\ \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots \\ \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots \\ \frac{3}{36} & \frac{4}{36} & \frac{5}{36} & \frac{6}{36} & \frac{5}{36} & \frac{4}{36} & \ddots & \ddots & \frac{1}{36} & \frac{2}{36} \\ \frac{2}{36} & \frac{3}{36} & \frac{4}{36} & \frac{5}{36} & \frac{6}{36} & \frac{5}{36} & \ddots & \ddots & 0 & \frac{1}{36} \\ \frac{1}{36} & \frac{2}{36} & \frac{3}{36} & \frac{4}{36} & \frac{5}{36} & \frac{6}{36} & \ddots & \ddots & 0 & 0 \end{bmatrix}$$

Deuxième modélisation du Monopoly

Hypothèses :

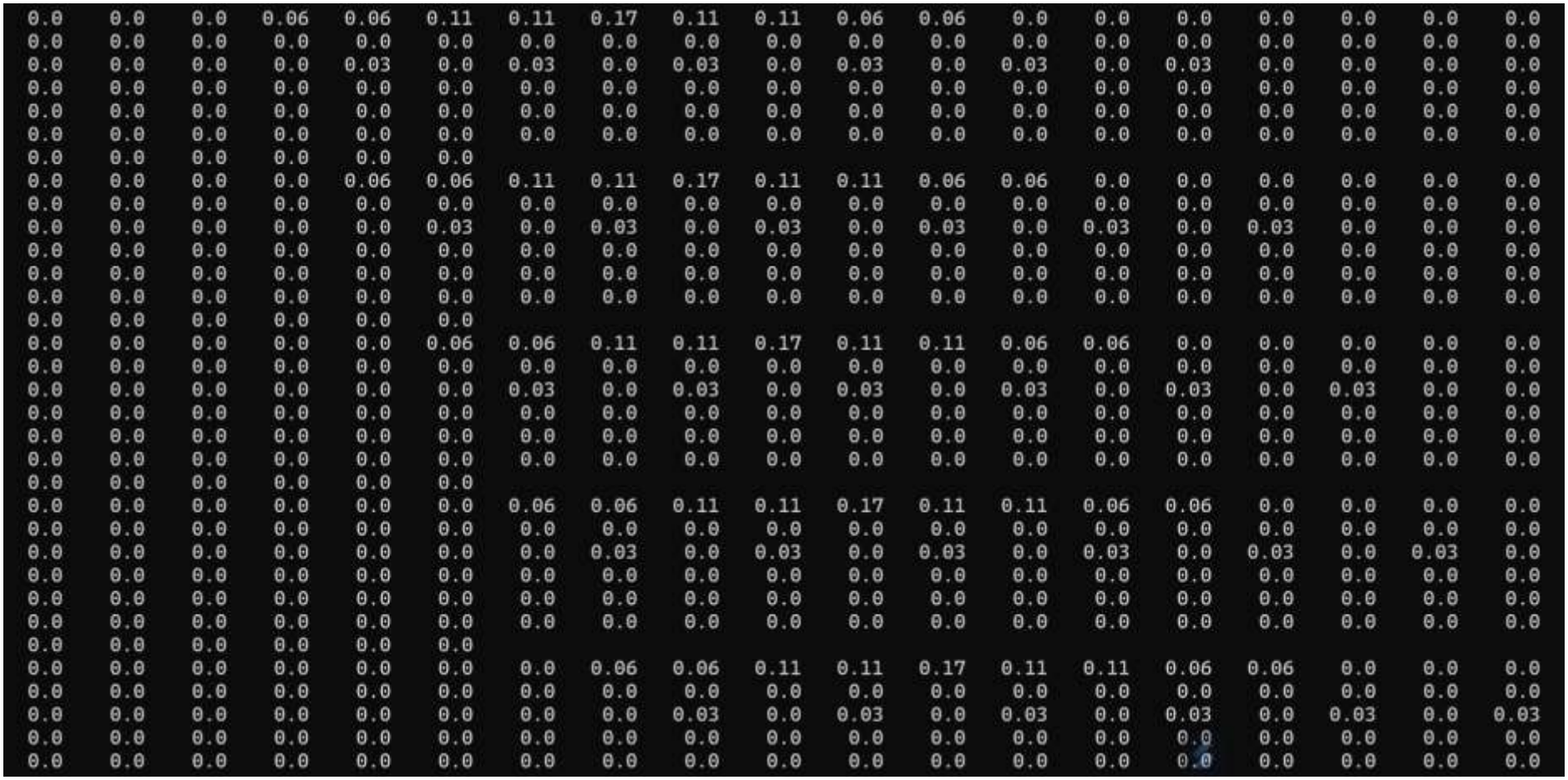
1. Pas de cartes Chance et Caisse de communauté
2. Séjour rapide en prison : sortie dès le tour suivant

Nouvel espace des états : $E = [0,39] \times [0,2]$ $\quad$ (Card(E)=120)

$$P = \begin{bmatrix} S & D & 0 \\ S & 0 & D \\ S + \sum_{i=1}^{40} \frac{1}{6} E_{i+80,30} & 0 & 0 \end{bmatrix}$$

Matrice de transition de la deuxième modélisation

33 premières lignes et 19 premières colonnes de la matrice de transition

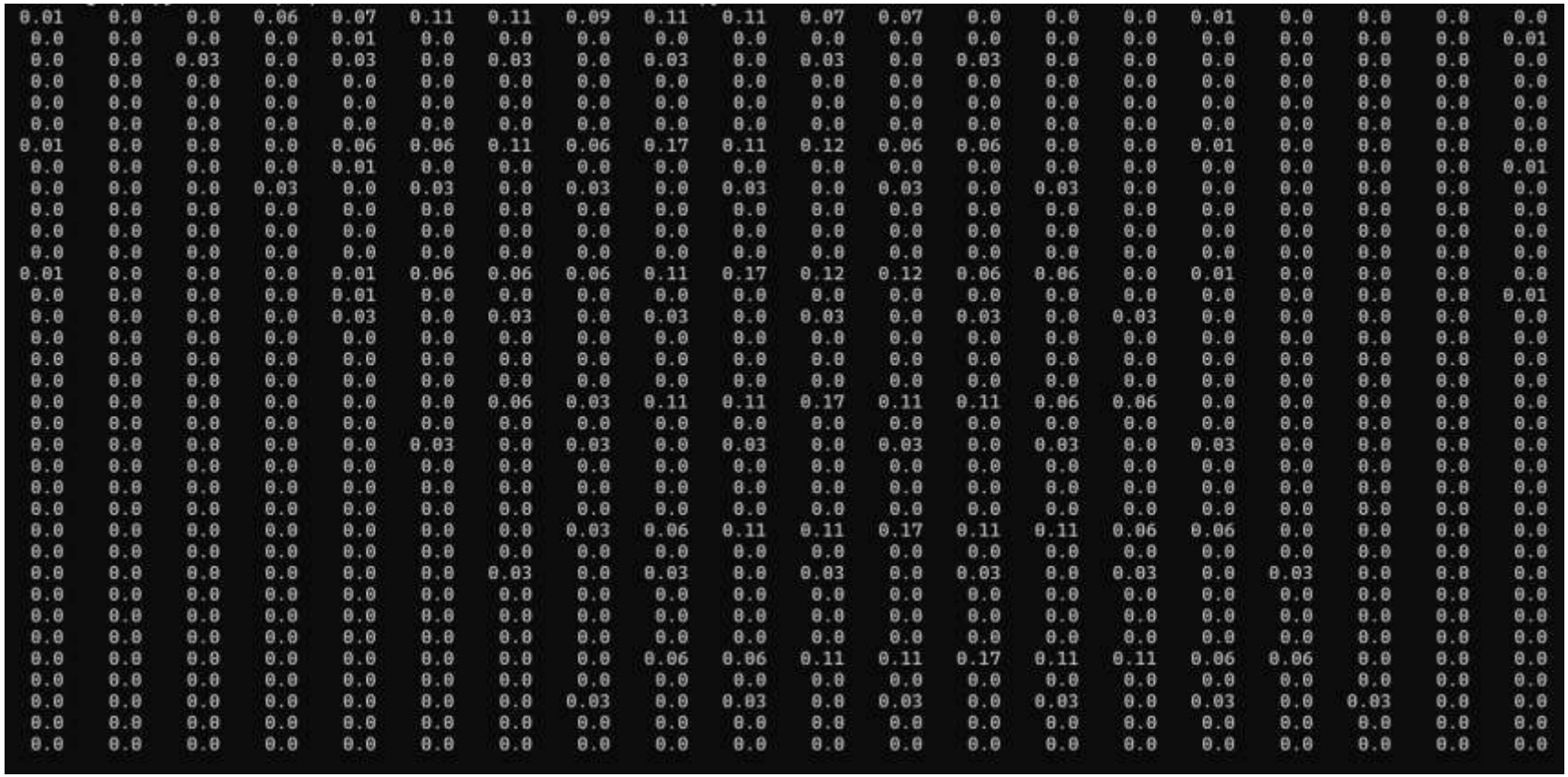


0.0	0.0	0.0	0.06	0.06	0.11	0.11	0.17	0.11	0.11	0.06	0.06	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.03	0.0	0.03	0.0	0.03	0.0	0.03	0.0	0.03	0.0	0.03	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0													
0.0	0.0	0.0	0.0	0.06	0.06	0.11	0.11	0.17	0.11	0.11	0.06	0.06	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.03	0.0	0.03	0.0	0.03	0.0	0.03	0.0	0.03	0.0	0.03	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0													
0.0	0.0	0.0	0.0	0.0	0.06	0.06	0.11	0.11	0.17	0.11	0.11	0.06	0.06	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.03	0.0	0.03	0.0	0.03	0.0	0.03	0.0	0.03	0.0	0.03	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0													
0.0	0.0	0.0	0.0	0.0	0.0	0.06	0.06	0.11	0.11	0.17	0.11	0.11	0.06	0.06	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.03	0.0	0.03	0.0	0.03	0.0	0.03	0.0	0.03	0.0	0.03	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

Troisième modélisation du Monopoly

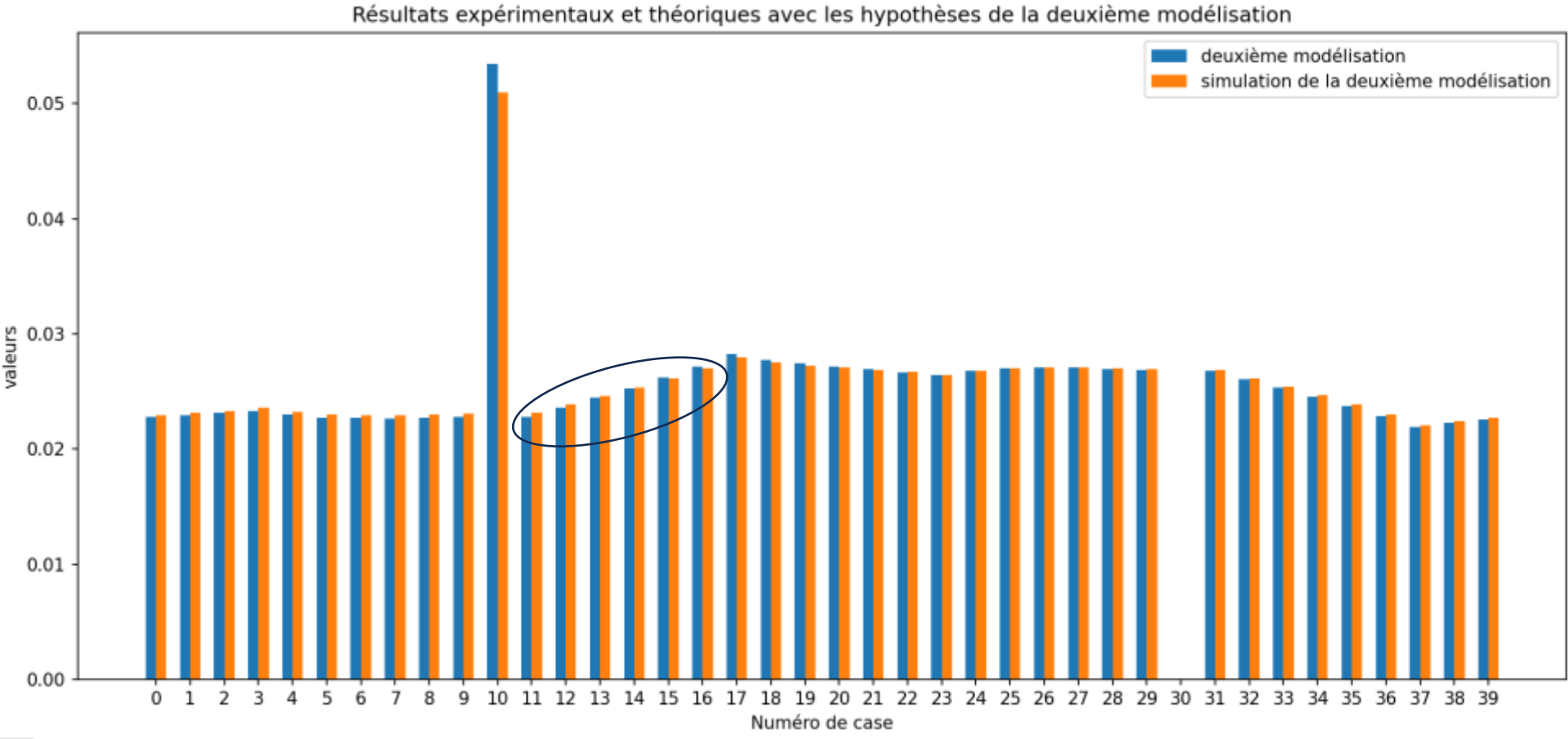
Hypothèses :

1. La pioche est remélangée avec remise à chaque fois qu'une carte est tirée
2. Séjour rapide en prison : sortie dès le tour suivant

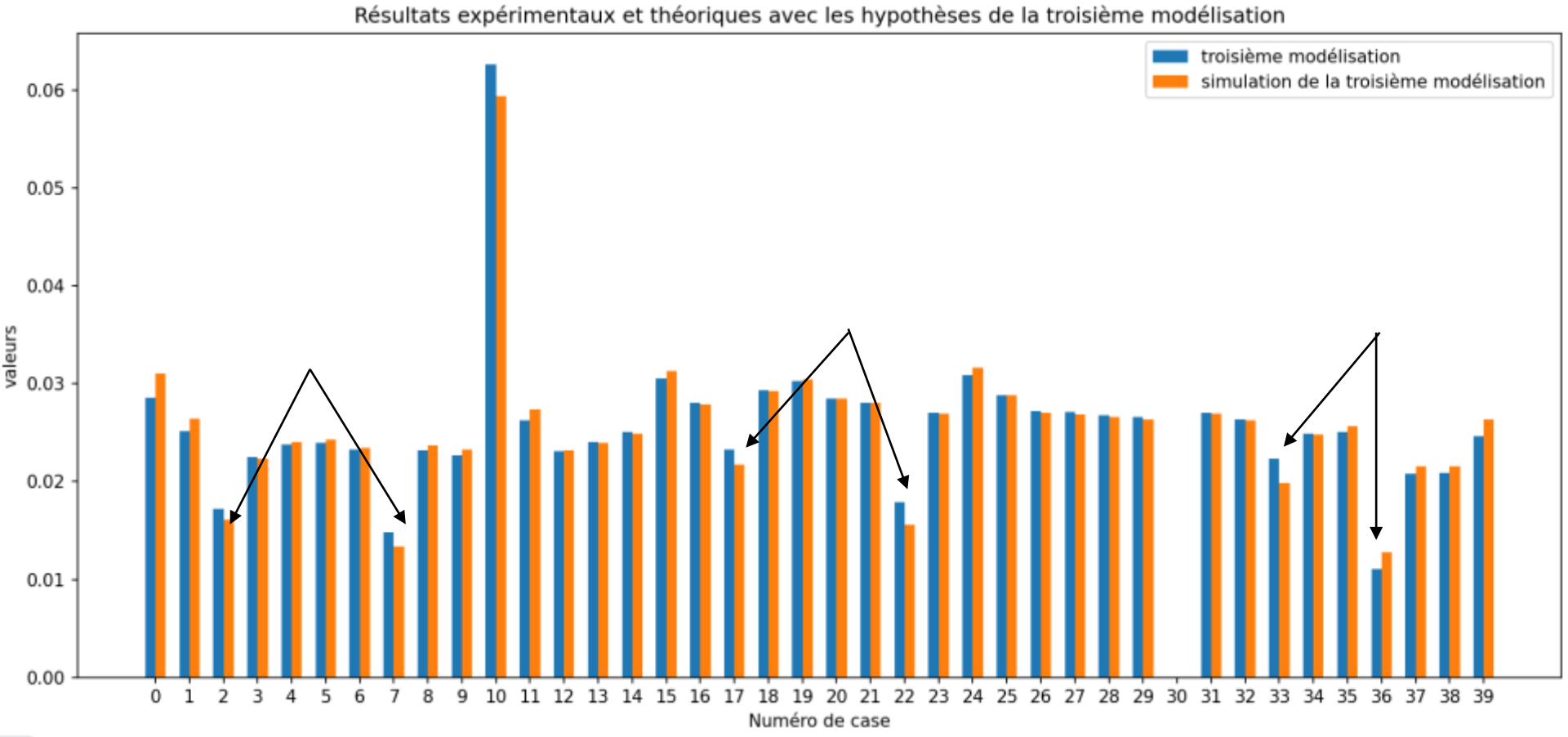


c) Simulation informatique

Résultats de la deuxième modélisation

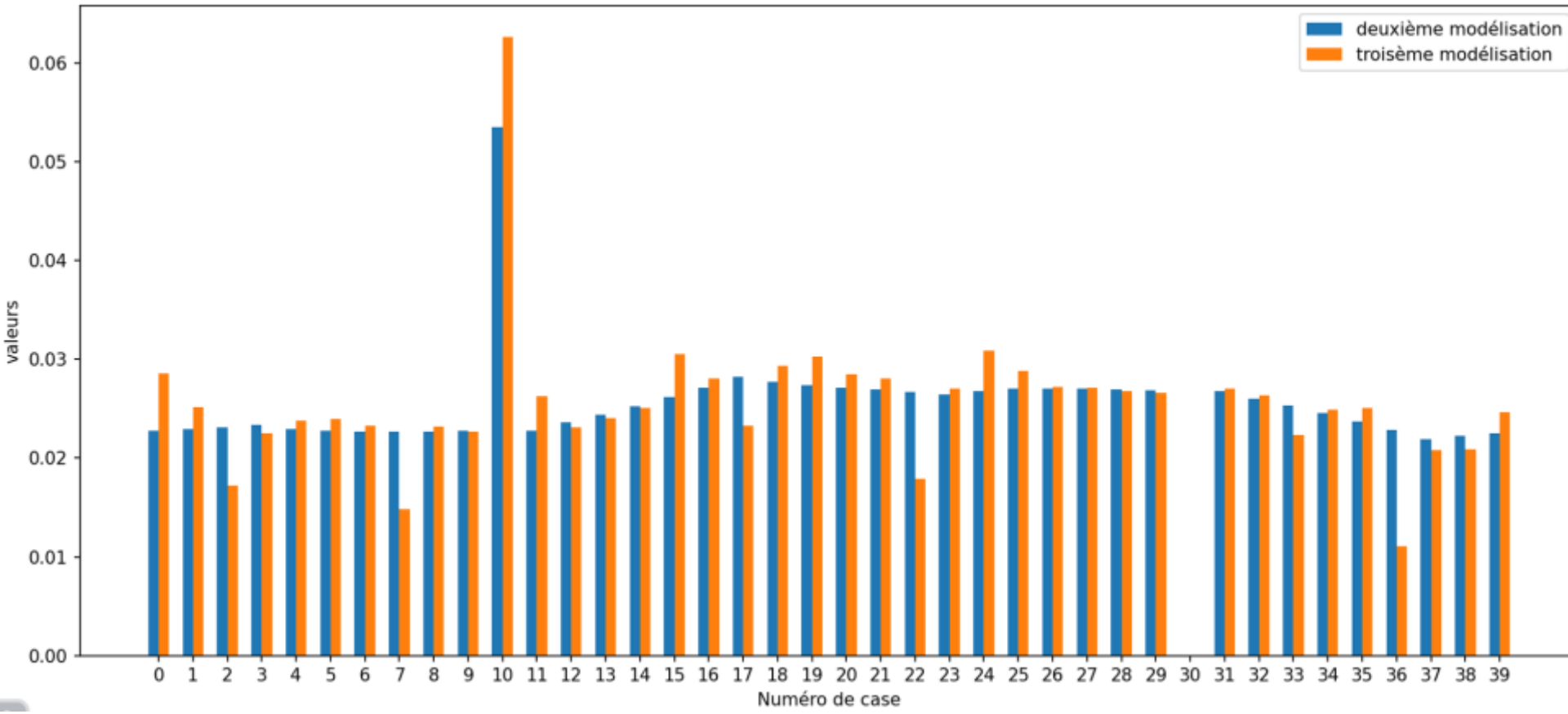


Résultats de la troisième modélisation



Importance des cartes Chance/Caisse de communauté

Résultats théoriques des deuxième et troisième modélisations



B. Aspects financiers du jeu

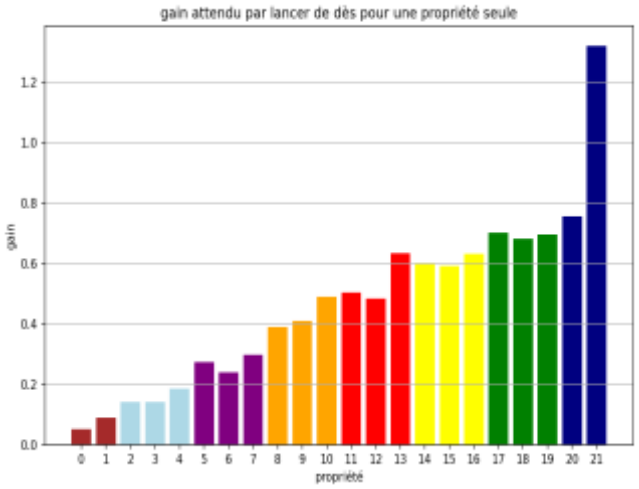
a) Gain par propriété

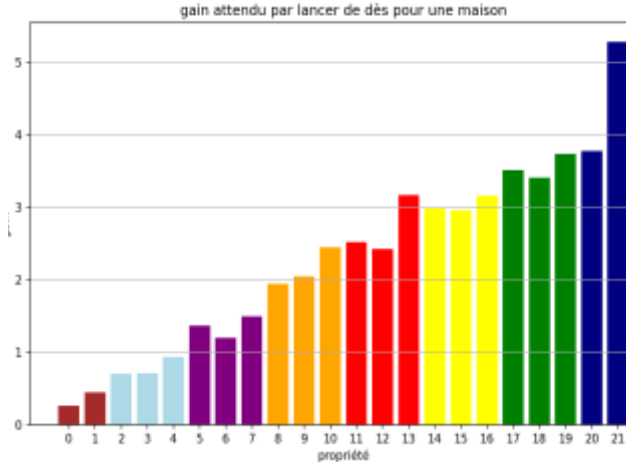
Gain attendu par lancers de dés : Gain que l'on peut s'attendre à gagner sachant que l'on possède la propriété et les constructions données.

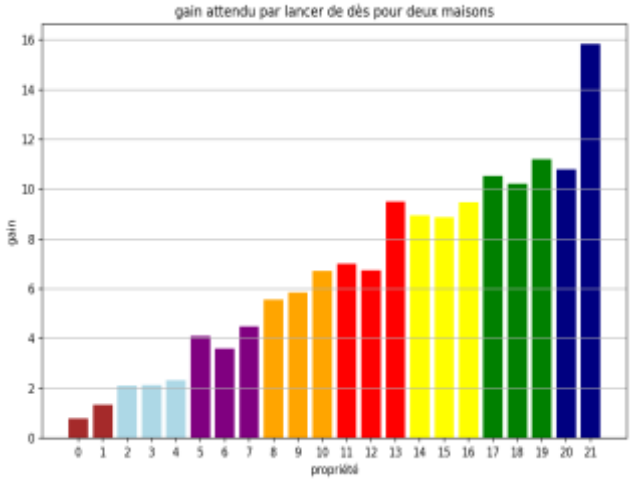
Propriété	propriété simple	une maison	deux maisons	3 maisons	4 maisons	hôtel
Boulevard de Belleville	0,0526004	0,263002	0,789006	2,367018	4,208032	6,57505
Rue Lecourbe	0,0897844	0,448922	1,346766	4,040298	7,182752	10,100745
Rue Vaugirard	0,141237	0,706185	2,118555	6,355665	9,4158	12,946725
Rue de Courcelles	0,142689	0,713445	2,140335	6,421005	9,5126	13,079625
Avenue de la République	0,1864936	0,932468	2,33117	6,99351	10,490265	13,98702
Boulevard de la Villette	0,27398	1,3699	4,1097	12,3291	17,12375	20,5485
Avenue de Neuilly	0,239629	1,199145	3,597435	10,792305	14,9893125	17,987175
Rue de Paradis	0,2989752	1,494876	4,484628	12,4573	17,44022	22,42314
Avenue Mozart	0,3895514	1,947757	5,56502	15,303805	20,868825	26,433845
Boulevard Saint-Michel	0,4096274	2,048137	5,85182	16,092505	21,944325	27,796145
Place Pigalle	0,489232	2,44616	6,72694	18,3462	24,4616	30,577
Avenue Matignon	0,504153	2,520765	7,002125	19,60595	24,5074375	29,408925
Boulevard Malesherbes	0,4849146	2,424573	6,734925	18,85779	23,5722375	28,286685
Avenue Henri-Martin	0,633492	3,16746	9,50238	23,75595	29,299005	34,84206
Faubourg Saint-Honoré	0,5973308	2,986654	8,959962	21,72112	26,472615	31,22411
Place de la Bourse	0,5914018	2,957009	8,871027	21,50552	26,2098525	30,914185
Rue La Fayette	0,6318048	3,159024	9,477072	22,37842	26,98333	31,59024
Avenue de Breteuil	0,7022886	3,511443	10,534329	24,30999	29,71221	34,4391525
Avenue Foch	0,682266	3,41133	10,23399	23,6169	28,8651	33,457275
Boulevard des Capucines	0,686668	3,73215	11,19645	24,881	29,8572	34,8334
Avenue des Champs-Elysée	0,755412	3,77706	10,7916	23,74152	28,05816	32,3748
Rue de la Paix	1,31999	5,27996	15,83988	36,95972	44,87966	52,7996

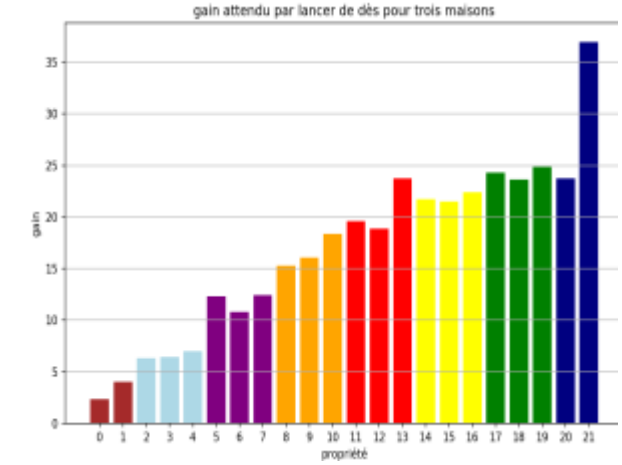
Gain attendu par lancer de dés

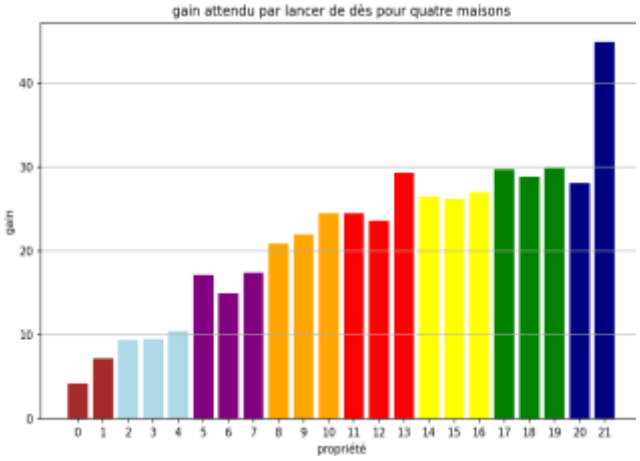
Simulations par propriété en fonction du nombre de constructions

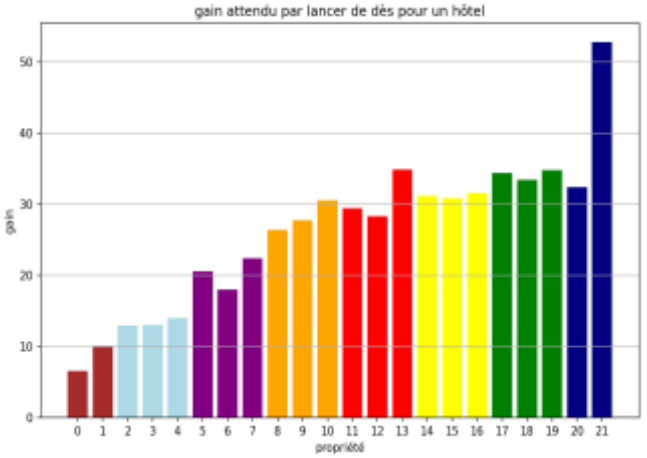










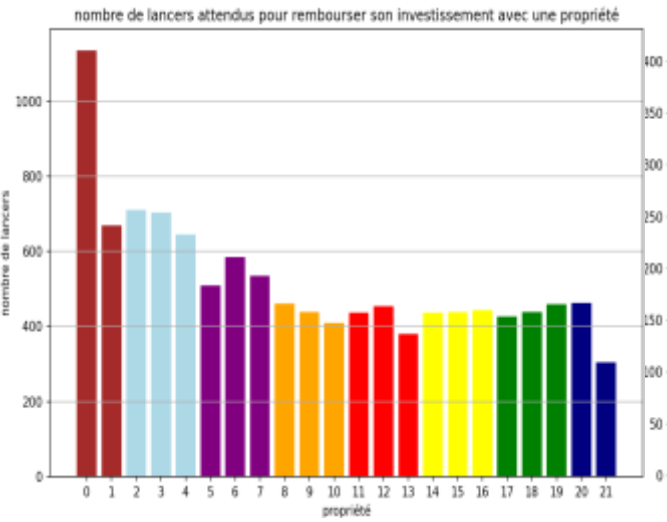


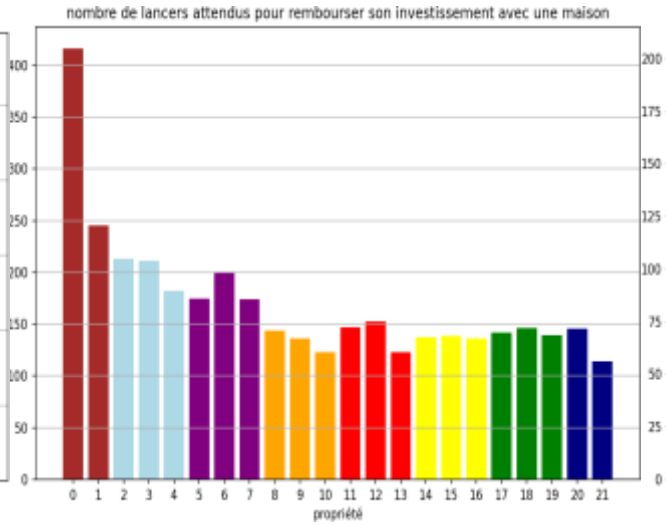
Temps mis pour obtenir un retour sur investissement

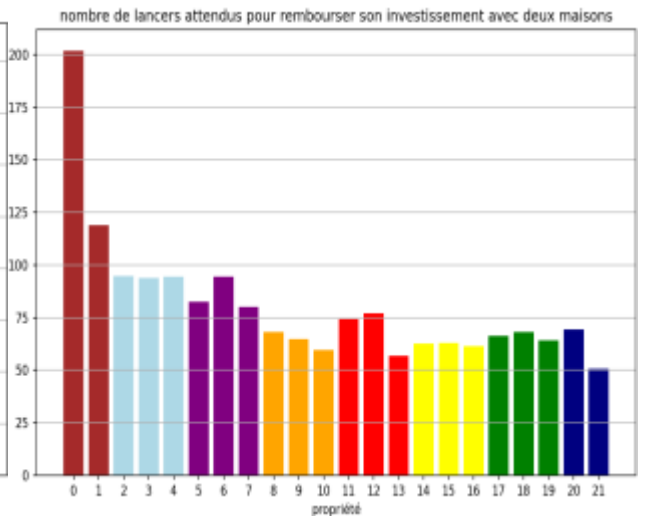
Étant donné un investissement réalisé (coût de la propriété et des constructions), nombre de lancers de dés adverses avant de pouvoir rembourser son investissement

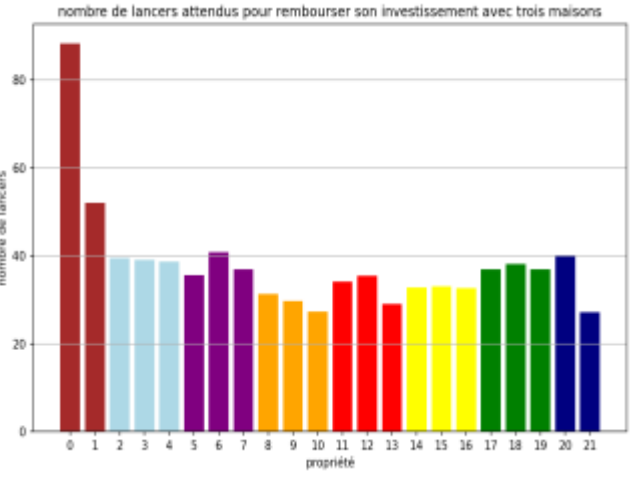
Propriété	propriété simple	une maison	deux maisons	3 maisons	4 maisons	hôtel
Boulevard de Bellevile	1135,241296	416,2551417	201,8206748	88,29654521	61,49223684	46,92330688
Rue Lecourbe	668,4700503	245,1056851	118,8391201	51,99211502	36,20879439	30,70010601
Rue Vaugirard	711,2324948	213,3697484	94,83099931	39,51291638	32,00546227	27,1561498
Rue de Courcelles	703,9061158	211,1718347	93,85414878	39,10589532	31,67577521	26,87641533
Avenue de la République	644,263478	182,5413188	94,49197677	38,65580868	30,54286118	26,48638743
Boulevard de la Villette	509,05944	174,5346651	82,41914742	35,55335771	31,41623972	31,02838491
Avenue de Neuilly	584,5462668	200,4158629	94,64082415	40,82545356	36,07485532	35,62948674
Rue de Paradis	534,7665076	173,799115	80,21497614	36,89888902	32,08599045	29,41215792
Avenue Mozart	461,5308877	143,5873873	68,20400896	31,32815723	27,76022821	25,69458509
Boulevard Saint-Michel	438,2225499	136,3359044	64,75955459	29,74601551	26,35827485	24,39695131
Place Pigalle	409,7057657	122,9117297	59,59356592	27,31371771	24,58234594	22,94352288
Avenue Matignon	436,0578625	146,6740083	74,20911987	34,14842741	33,43487818	32,9591787
Boulevard Malesherbes	453,338114	152,4864565	77,14990449	35,50167308	34,75984708	34,26529641
Avenue Henri-Martin	379,3398854	123,2854627	56,90098281	29,08272455	28,70680214	28,4504914
Faubourg Saint-Honoré	436,4092915	137,6367765	62,66389827	32,77265929	32,57145481	32,43148648
Place de la Bourse	439,3261818	138,5567189	63,08273379	32,99170654	32,78915724	32,64825338
Rue Lafayette	444,109459	136,4050481	61,32940148	32,6924274	32,68150444	32,67376734
Avenue de Breteuil	426,3351182	142,1117061	66,31879616	36,94904357	36,94904357	37,67353462
Avenue Foch	439,3512247	146,4504082	68,34352384	38,07710614	38,07710614	38,82371606
Boulevard des Capucines	459,3200341	139,327077	64,30480477	36,97526275	37,51113612	37,89390281
Avenue des Champs-Elysé	463,2310365	145,5868972	69,48465547	40,00631679	40,97813015	41,69079328
Rue de la Paix	303,9317369	113,9744013	50,65528949	27,13676223	26,8175062	26,59402698

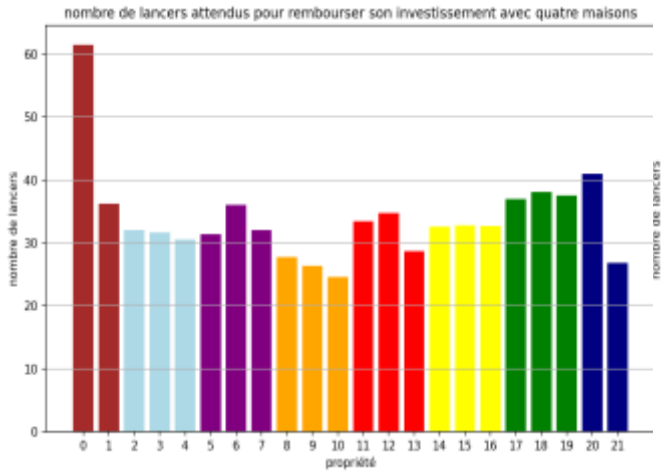
b) Optimisation des dépenses dans le Monopoly

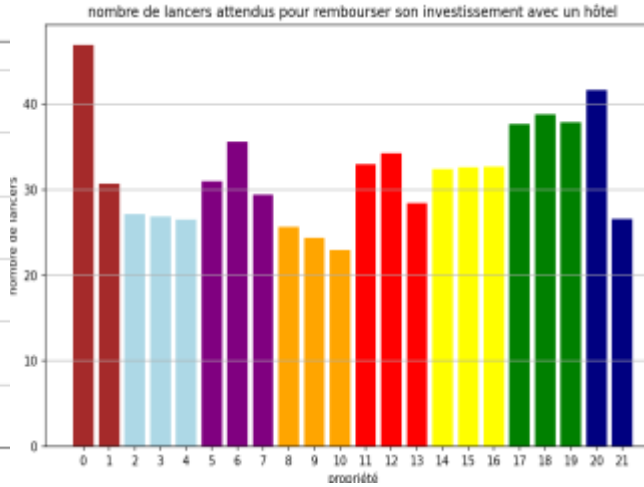




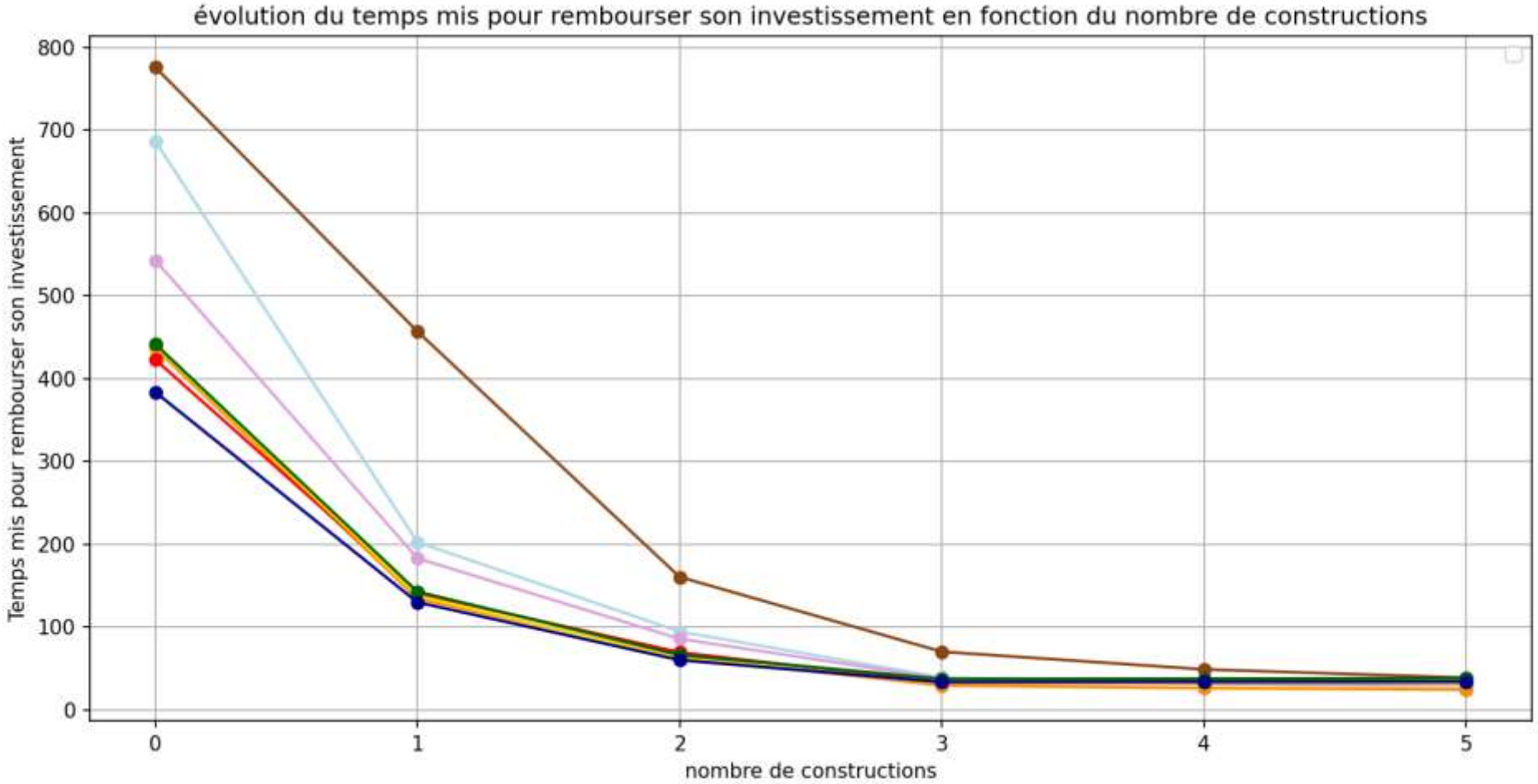








Combien de maisons faut-il acheter ?



c)Temps séjour court ou long ?

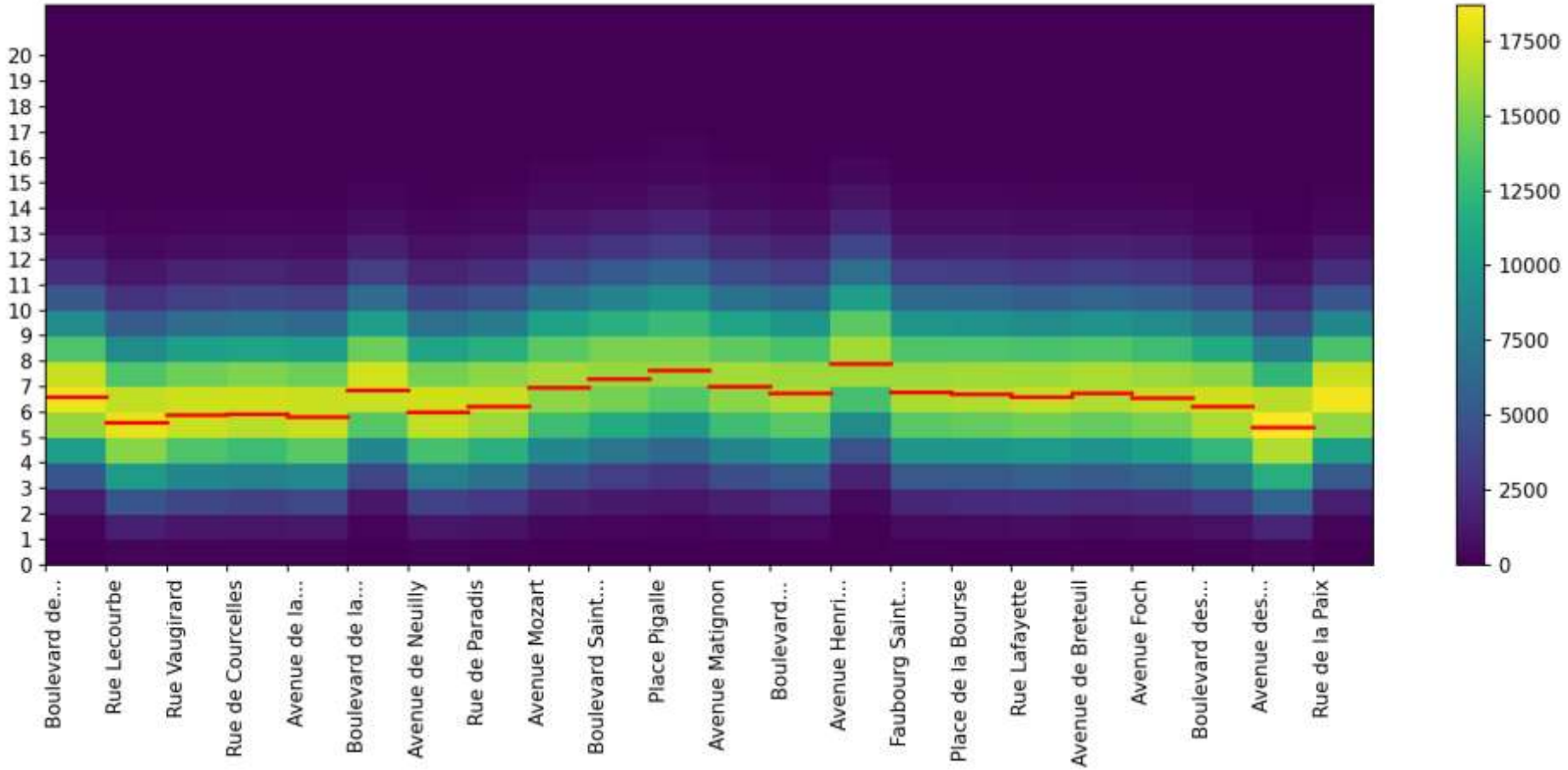
Case départ	Séjour rapide
Case départ	33,01
Impôts	-4,801
Taxe de luxe	-1,889
Caisse de communauté	1,469
Chance	0,852
Total	28,641

Gain attendu par lancer de dés pour certaines cases remarquables du plateau.

0,263002	0,789006	2,367018
0,448922	1,346766	4,040298
0,706185	2,118555	6,355665
0,713445	2,140335	6,421005
0,932468	2,33117	6,99351
1,3699	4,1097	12,3291
1,199145	3,597435	10,792305
1,494876	4,484628	12,4573
1,947757	5,56502	15,303805
2,048137	5,85182	16,092505
2,44616	6,72694	18,3462
2,520765	7,002125	19,60595
2,424573	6,734925	18,85779
3,16746	9,50238	23,75595
2,986654	8,959962	21,72112
2,957009	8,871027	21,50552
3,159024	9,477072	22,37642
3,511443	10,534329	24,30999
3,41133	10,23399	23,6169
3,73215	11,19645	24,881
3,77706	10,7916	23,74152
5,27996	15,83988	36,95972

Gain attendu par lancer de dés pour les propriétés avec une, deux et trois maisons.

Validité des résultats sur des parties courtes (250 lancers de dés)



ANNEXE

A. Monopoly

1) Matrices D et S
2) Matrices de transition des deuxième et troisième modélisations
3) Affichage de la matrice dans le terminal
4) Code Monopoly

B. Files d'attente et théorie

1) Propriétés de la matrice stochastique (temps continu et temps discret)
2) Générateur de processus infinitésimal
3) Théorème de Little et application à la file M/M/1 dans le cas du débit
4) Régime permanent et distribution stationnaire
5) Les programmes

Matrice D

```
import numpy as np

D=np.zeros((40,40)) #matrice décrivant les probabilités de déplacement avec uniquement des doubles

for i in range (0,30):
    for k in range (1,7):
        D[i][(i+2*k)%40]=1/36

for i in range (31,40):
    for k in range (1,7):
        D[i][(i+2*k)%40]=1/36

for k in range (1,7):
    D[30][(10+2*k)%40]=1/36
```

Matrice S

```
S=np.zeros((40,40)) #matrice décrivant les probabilités de déplacements sans les doubles

for i in range (0,30):
    S[i][(i+3)%40]=1/18
    S[i][(i+4)%40]=1/18
    S[i][(i+5)%40]=1/9
    S[i][(i+6)%40]=1/9
    S[i][(i+7)%40]=1/6
    S[i][(i+8)%40]=1/9
    S[i][(i+9)%40]=1/9
    S[i][(i+10)%40]=1/18
    S[i][(i+11)%40]=1/18

for i in range (31,40):
    S[i][(i+3)%40]=1/18
    S[i][(i+4)%40]=1/18
    S[i][(i+5)%40]=1/9
    S[i][(i+6)%40]=1/9
    S[i][(i+7)%40]=1/6
    S[i][(i+8)%40]=1/9
    S[i][(i+9)%40]=1/9
    S[i][(i+10)%40]=1/18
    S[i][(i+11)%40]=1/18

S[30][(10+3)%40]=1/18 #lorsqu'on est à la case prison on repart de la case 10
S[30][(10+4)%40]=1/18
S[30][(10+5)%40]=1/9
S[30][(10+6)%40]=1/9
S[30][(10+7)%40]=1/6
S[30][(10+8)%40]=1/9
S[30][(10+9)%40]=1/9
S[30][(10+10)%40]=1/18
S[30][(10+11)%40]=1/18
```

Matrice de transition pour la deuxième modélisation

```
# N est la matrice de transition avec la régle des 3 doubles et sans les cartes chances et caisse de
communauté
N=np.zeros((120,120))

for k in range (0,120): #On sélectionne l'état d'arrivée
    for i in range (0,120):#on sélectionne létat de départ
        if k <=39 and k!=30 : # 0, 1 ou 2 doubles au départ et on veut 0 double à l'arrivée
            N[i][k]= S[i%40][k%40]
        elif k==30 :#Il faut traiter le cas où on arrive à la case prison : on peut soit tommber
dessus soit faire un double
            if i<=79:
                N[i][k]=S[i%40][k%40]
            else :
                N[i][k]=1/6 +S[i%40][k%40]
        elif k>=40 and k<=79: #on veut 1 double à l'arrivée
            if i<= 39 and i!=30: # on est à 0 double et on doit passer à un double
                N[i][k]=D[i%40][k%40] #on ne peut que faire un double
            elif i>=40 and i<=79 : #on fait un double et on veut passer à un double : impossible
                N[i][k]=0
            elif i>=80: # on fait deux doubles et on veut passer à un double : impossible
                N[i][k]=0
            else:
                N[i][k]=D[30][k%40]#Cas i=30?
        elif k>=80 : #on veut deux doubles à l'arrivée
            if i<= 39 :
                N[i][k]=0
            elif i>= 40 and i<=79: #on a un double et on veut deux à l'arrivée
                N[i][k]=D[i%40][k%40]
            else :
                N[i][k]=0
```

Matrice de transition pour la troisième modélisation

```
#P est la matrice de transition en prenant en compte tout les aspects du jeu
#Cases remarquables qui influent sur les déplacements : allez en prison(30), carte chance(7, 22, 36),
carte caisse de communauté (2,17,33) : on peut donc reproduire la matrice suivante N et rajouter ces
cas particuliers
#pour une carte chance le modèle adopter est le suivant : On suppose que la probabilité de piocher une
carte cchance est la mm pour toutes (sous peine de devoir remodifier notre espace des états)
#pour une des cartes chance il y a une carte qui nécéssite une décision du joueur : le choix entre
piocher une carte caisse de communauté ou de payer une amende. On supposera cependant que le joueur
paye l'amende

P = N.copy()  # Crée une copie de N pour éviter les modifications in-place
for i in range(0, 120):
    for k in range(0,120):
        z=P[i][k]
        if k == 7%40 or k == 22%40 or k == 33%40:# carte chance
            P[i][k] += -(7/16 * z)
            P[i][39] += 1/16 * z
            P[i][0] += 1/16 * z
            P[i][24] += 1/16 * z
            P[i][11] += 1/16 * z
            P[i][15] += 1/16 * z
            P[i][10] += 1/16 * z
            P[i][(k-3) % 40] += 1/16 * z
        if k == 2%40 or k == 17%40 or k == 33%40:  # carte caisse de communauté
            P[i][k] += -(4/16 * z)
            P[i][0] += 1/16 * z
            P[i][10] += 1/16 * z
            P[i][1] += 1/16 * z
            if k == 2%40:
                P[i][5] += 1/16 * z
            if k == 17%40:
                P[i][25] += 1/16 * z
            if k == 33%40:
                P[i][35] += 1/16 * z

        # On traite le cas où la carte caisse de communauté fait avancer le joueur jusqu'à la gare la
plus proche
```

Affichage de la matrice dans le terminal

```
def show(R):
    T = []
    for l in R:
        T.append([])
        for c in l:
            T[-1].append(round(c, 2))
    print('\n'.join([''.join(['{:7}'.format(c) for c in l]) for l in T]))
```

Code Monopoly

```
from math import *
from random import *
import matplotlib.pyplot as plt
import numpy as np

case=[i for i in range(40)]
carte_chance=["carte1","carte2","carte3","carte4","carte5","carte6","carte7","carte8","carte9","carte10","carte11","c
arte12","carte13","carte14","carte15","carte16"]
caisse_communauté3=["caisse1","caisse2","caisse3","caisse4","caisse5","caisse6","caisse7","caisse8","caisse9","caisse
10","caisse11","caisse12","caisse13","caisse14","caisse15","caisse16"]
def lancer():
    # cette fonction simule le lancer de deux dés et en fait la somme
    #La variable de est un entier de 1 à 6. Elle repréente le résultat du premier dé.
    de1=randint(1,6)
    de2=randint(1,6)
    return de1+de2



def avance(k,L):
    #cette fonction simule l'avancée du joueur se trouvant à la case k après les lancer de dé. On ne tient pas compte
de la case prison
    #La variable k est un entier de 0 à 40 qui représente la position du joueur
    #L est la liste des fréquences du monopoly qui sont incrémentées quand on passe dessus

    k=k+lancer()    #le joueur avance de "lancer" cases
    k=k%40          #On fait une division euclidienne pour revenir à 0 après la case 39
    L[k]+=1
    return k
```

```
def listedescases(nbtours):
    #cette fonction compte pour chaque case du monopoly le nombre de fois où le joueur est tombé sur la case.
    #La variable nbtours est un entier donnat le nombre de tours effectuées
    # la variablerenvoyée L est une liste de 40 valeurs. Pour tout 0<=i<=39 L[i] représente le nombre de fois où le
joeur est tombé sur case[i]
    #La variable j est un entier qui désigne le numéro du tour
    # la vatiable k est un entier entre 0 et 40 représentant la position du joueur après le tour j

    L=[0 for i in range(40)]           # on initialise toutes les cases à 0
    k=0                                # on initialise la position du joueur
    melange(carte_chance)
    melange(caisse_communauté3)         # on mélange la liste
    for j in range (nbtours) :         #on joue "nbtours"  tours
        k=avanceP(k,L)                 # on avance et on incrémente L au fur et à mesure
    return L


def frequence(nbtours):
    #cette fonction compte pour chaque case du monopoly la fréquence de la case sur nbtours
    #La variable nbtours est un entier donnat le nombre de tours effectuées
    # la variablerenvoyée L est une liste de 41 valeurs. Pour tout 0<=i<=40 L[i] la fréquence case[i]
    #La variable j est un entier qui désigne le numéro du tour
    # la vatiable k est un entier entre 0 et 40 représentant la position du joueur après le tour j

    L=listedescases(nbtours) # on calcule la liste des effectifs
    for i in range(40):
        L[i]=L[i]/nbtours       # on divise par l'efectif total
    return L

def nombre_de_tours(nbtours):
    G=[]
    k=0
    1

def melange(M):
    shuffle(M)
```

```
def chance(k,carte_chance):

    if k==36 or k==7 or k==22 :
        if carte_chance[0]== "carte1":
            k=39

        elif carte_chance[0]=="carte2":
            k=0

        elif carte_chance[0]=="carte3":
            k=24

        elif carte_chance[0]=="carte4":
            k=11

        elif carte_chance[0]=="carte5":
            k=k

        elif carte_chance[0]=="carte6":
            k=15

        elif carte_chance[0]=="carte7":
            k=k

        elif carte_chance[0]=="carte8":
            k=k

        elif carte_chance[0]=="carte9":
            k=k

        elif carte_chance[0]== "carte10":      #Carte chance qui recule le joueur de 3 case 1/16
            k=k-3
            k=k%40

        elif carte_chance[0]== "carte11":
            k=10

        elif carte_chance[0]=="carte12":
            k=k
```

```
        elif carte_chance[0]== "carte11":
            k=10

        elif carte_chance[0]=="carte12":
            k=k

        elif carte_chance[0]=="carte13":
            k=k

        elif carte_chance[0]=="carte14":
            k=k

        elif carte_chance[0]=="carte15":
            k=k

        else :
            k=k

        carte_chance.append(carte_chance[0])
        del carte_chance[0]   #on met la dernière carte en bas du paquet
    return k
```

```
def caisse_communauté2(k, caisse_communauté3):
    if k==2 or k==17 or k==33 :
        if caisse_communauté3[0]=="caisse1":
            k=0

        elif caisse_communauté3[0]=="caisse2":
            k=k

        elif caisse_communauté3[0]=="caisse3":
            k=k

        elif caisse_communauté3[0]=="caisse4":
            k=k

        elif caisse_communauté3[0]=="caisse5":
            k=k

        elif caisse_communauté3[0]=="caisse6":
            k=10

        elif caisse_communauté3[0]=="caisse7":
            k=1

        elif caisse_communauté3[0]=="caisse8":
            k=k

        elif caisse_communauté3[0]=="caisse9":
            k=k

        elif caisse_communauté3[0]=="caisse10":
            k=k

        elif caisse_communauté3[0]=="caisse11":
            k=k

        elif caisse_communauté3[0]=="caisse12":
            k=k
```

```
        elif caisse_communauté3[0]=="caisse13":
            z=randint(1,2)
            if z==1:
                k=k
            elif z==2:
                if carte_chance[0]== "carte1":
                    k=39

                elif carte_chance[0]=="carte2":
                    k=0

                elif carte_chance[0]=="carte3":
                    k=24

                elif carte_chance[0]=="carte4":

                    k=11

                elif carte_chance[0]=="carte5":
                    k=k

                elif carte_chance[0]=="carte6":

                    k=15

                elif carte_chance[0]=="carte7":
                    k=k

                elif carte_chance[0]=="carte8":
                    k=k

                elif carte_chance[0]=="carte9":
                    k=k

                elif carte_chance[0]== "carte10":      #Carte chance qui recule le joueur de 3 case 1/16
                    k=k-3
                    k=k%40

                elif carte_chance[0]== "carte11":

                elif carte_chance[0]=="carte12":
                    k=k

                elif carte_chance[0]=="carte13":
                    k=k

                elif carte_chance[0]=="carte14":
                    k=k

                elif carte_chance[0]=="carte15":
                    k=k

                elif carte_chance[0]=="carte16":
                    k=k

                carte_chance.append(carte_chance[0])
                del carte_chance[0]

        if caisse_communauté3[0]=="caisse14":
            if k==2:
                k=5
            elif k==17:
                k=25
            elif k==33:
                k=35
        if caisse_communauté3[0]=="caisse15":
            k=k

        if caisse_communauté3[0]=="caisse16":
            k=k

        caisse_communauté3.append(caisse_communauté3[0])
        del caisse_communauté3[0]   #on met la dernière carte en bas du paquet
    return k
```

Annexe 10

```
#Loyer, avec une maison, 2 maisons,3,4, un hôtel, prix de la propriété, prix d'une maison)

Cproprietes_monopoly = [
    ["Boulevard de Belleville", 2, 10, 30, 90, 160, 250, 60, 50],
    ["Rue Lecourbe", 4, 20, 60, 180, 320, 450, 60, 50],
    ["Rue de Vaugirard", 6, 30, 90, 270, 400, 550, 100, 50],
    ["Rue de Courcelles", 6, 30, 90, 270, 400, 550, 100, 50],
    ["Avenue de la République", 8, 40, 100, 300, 450, 600, 120, 50],
    ["Boulevard de la Villette", 10, 50, 150, 450, 625, 750, 140, 100],
    ["Avenue de Neuilly", 10, 50, 150, 450, 625, 750, 140, 100],
    ["Rue de Paradis", 12, 60, 180, 500, 700, 900, 160, 100],
    ["Avenue Mozart", 14, 70, 200, 550, 750, 950, 180, 100],
    ["Boulevard Saint-Michel", 14, 70, 200, 550, 750, 950, 180, 100],
    ["Place Pigalle", 16, 80, 220, 600, 800, 1000, 200, 100],
    ["Avenue Matignon", 18, 90, 250, 700, 875, 1050, 220, 150],
    ["Boulevard Malesherbes", 18, 90, 250, 700, 875, 1050, 220, 150],
    ["Avenue Henri-Martin", 20, 100, 300, 750, 925, 1100, 240, 150],
    ["Faubourg Saint-Honoré", 22, 110, 330, 800, 975, 1150, 260, 150],
    ["Place de la Bourse", 22, 110, 330, 800, 975, 1150, 260, 150],
    ["Rue La Fayette", 24, 120, 360, 850, 1025, 1200, 280, 150],
    ["Avenue de Breteuil", 26, 130, 390, 900, 1100, 1275, 300, 200],
    ["Avenue Foch", 26, 130, 390, 900, 1100, 1275, 300, 200],
    ["Boulevard des Capucines", 28, 150, 450, 1000, 1200, 1400, 320, 200],
    ["Avenue des Champs-Élysées", 35, 175, 500, 1100, 1300, 1500, 350, 200],
    ["Rue de la Paix", 50, 200, 600, 1400, 1700, 2000, 400, 200]
]
```

```
PInEXP_test=np.zeros((10000,40))

for i in range (0,1000):
    PInEXP_test[i]=frequence(1000000)


PInEXPfinal=[np.mean([l[i] for l in PInEXP_test]) for i in range (len(PInEXP_test[0]))]
print(PInEXPfinal)
PINPROP=[]
PINPROP.append(PInEXPfinal[1])
PINPROP.append(PInEXPfinal[3])
PINPROP.append(PInEXPfinal[6])
PINPROP.append(PInEXPfinal[8])
PINPROP.append(PInEXPfinal[9])
PINPROP.append(PInEXPfinal[11])
PINPROP.append(PInEXPfinal[13])
PINPROP.append(PInEXPfinal[14])
PINPROP.append(PInEXPfinal[16])
PINPROP.append(PInEXPfinal[18])
PINPROP.append(PInEXPfinal[19])
PINPROP.append(PInEXPfinal[21])
PINPROP.append(PInEXPfinal[23])
PINPROP.append(PInEXPfinal[24])
PINPROP.append(PInEXPfinal[26])
PINPROP.append(PInEXPfinal[27])
PINPROP.append(PInEXPfinal[29])
PINPROP.append(PInEXPfinal[31])
PINPROP.append(PInEXPfinal[32])
PINPROP.append(PInEXPfinal[34])
PINPROP.append(PInEXPfinal[37])
PINPROP.append(PInEXPfinal[39])
```

```
def gain_par_lancer(C,k):
    G=np.zeros(22)
    for i in range (0,22):
        G[i]=(PINPROP[i]*C[i][k])
    G=list(G)
    return G

GG=[]
for j in range (1,7):
    G_test=[]
    Gfinal=[]
    for i in range (0,1000):
        V=[]
        V=gain_par_lancer(Cproprietes_monopoly,j)
        G_test.append(V)
    Gfinal=[np.mean([l[i] for l in G_test]) for i in range (len(G_test[0]))]
    GG.append(Gfinal)
```

```
def attendu_du_remboursement(C,j):
    R=np.zeros(22)
    N=PINPROP
    for i in range (0,22):
        R[i]=((j-1)*C[i][8]+C[i][7])/GG[j-1][i]
    R=list(R)
    return(R)
GA=[]
for j in range (1,7):
    GA_test=[]
    GAfinal=[]
    for i in range (0,1000):
        V=[]
        V=attendu_du_remboursement(Cproprietes_monopoly,j)
        GA_test.append(V)
    GAfinal=[np.mean([l[i] for l in GA_test]) for i in range (len(GA_test[0]))]
    GA.append(GAfinal)
```

```python
import numpy as np
def calcul_vecteur_PIn(N):
    PI0=np.zeros(120)
    PIn=np.zeros(120)
    PI0[0]=1
    Q=np.linalg.matrix_power(N,1000000)
    for j in range (0,120):
        a=0
        for i in range (0,120):
            a+=PI0[i]*Q[i][j]
        PIn[j]=a
    return PIn


def proba_case(A):
    Pn=np.zeros(40)
    for i in range (0,40):
        Pn[i]=A[i]+A[i+40]+A[i+80]
    Pn[10]+=Pn[30]
    Pn[30]=0
    Pn=list(Pn)
    return Pn

Npin=proba_case(calcul_vecteur_PIn(N))
Ppin=proba_case(calcul_vecteur_PIn(P))
```

```python
import matplotlib.pyplot as plt
import numpy as np


num_cases = list(range(0, 40))
th2 = Npin
th3 = Ppin
exp= [0.022944359, 0.023117624000000003, 0.023309638, 0.023563751, 0.023198048, 0.023011765999999996,
0.022941074000000002, 0.022952485999999998, 0.023003622, 0.0230772, 0.04998769200000001, 0.023142581000000002,
0.023859712999999998, 0.024581449999999998, 0.025338932999999998, 0.026128969999999998, 0.026997243000000004,
0.027969232, 0.027548764, 0.027257749999999997, 0.027065681, 0.026881160999999997, 0.02669517, 0.02642368,
0.026801461000000002, 0.026995778999999998, 0.027056706000000003, 0.027056424, 0.026998689000000003,
0.026924649000000005, 0.0, 0.026858930999999996, 0.026132446000000004, 0.025419467, 0.024663163000000002,
0.023875904, 0.023000806, 0.022034391, 0.02244907, 0.022734525999999998]
bar_width = 0.3
index = np.arange(len(num_cases))

fig, ax = plt.subplots(figsize=(12, 6))

bar1 = plt.bar(index, th2, bar_width, label='théorie')
bar2 = plt.bar(index , th2, bar_width, label='deuxième modélisation')
bar3 =plt.bar(index + bar_width, th3, bar_width, label='troisème modélisation')

plt.xlabel('Numéro de case')
plt.ylabel('valeurs')
plt.title('Résultats théoriques de la deuxième et troisième modélisation')
plt.xticks(index + bar_width / 2, num_cases)
plt.legend()
plt.tight_layout()
plt.show()
```

```python
import matplotlib.pyplot as plt
import numpy as np

# Fonction pour calculer la moyenne des valeurs de chaque sous-liste
def calculate_means(data):
    means = [np.mean(sublist) for sublist in data]
    return means

# Données
data1 = [
    [1135.241296, 416.2551417],
    [668.4700503, 245.1056851],
    [201.8206748, 118.8391201],
    [88.29654521, 51.99211502],
    [61.49223684, 36.20879439],
    [46.92330688, 30.70010601]
]

data2 = [
    [711.2324948, 703.9061158, 644.263478],
    [213.3697484, 211.1718347, 182.5413188],
    [94.83099931, 93.85414878, 94.49197677],
    [39.51291638, 39.10589532, 38.65580868],
    [32.00546227, 31.67577521, 30.54286118],
    [27.1561498, 26.87641533, 26.48638743]
]

data3 = [
    [509.05944, 584.5462668, 534.7665076],
    [174.5346651, 200.4158629, 173.799115],
    [82.41914742, 94.64082415, 80.21497614],
    [35.55335771, 40.82545356, 36.89888902],
    [31.41623972, 36.07485532, 32.08599045],
    [31.02838491, 35.62948674, 29.41215792]
]

data4 = [
    [461.5308877, 438.2225499, 409.7057657],
    [143.5873873, 136.3359044, 122.9117297],
    [68.20400896, 64.75955459, 59.59356592],
    [31.32815723, 29.74601551, 27.31371771],
    [27.76022821, 26.35827485, 24.58234594],
    [25.69458509, 24.39695131, 22.94352288]
]

data5 = [
    [436.0578625, 453.338114, 379.3398854],
    [146.6740083, 152.4864565, 123.2854627],
    [74.20911987, 77.14990449, 56.90098281],
    [34.14842741, 35.50167308, 29.08272455],
    [33.43487818, 34.75984708, 28.70680214],
    [32.9591787, 34.26529641, 28.4504914]
]

data6 = [
    [436.4092915, 439.3261818, 444.109459],
    [137.6367765, 138.5567189, 136.4050481],
    [62.66389827, 63.08273379, 61.32940148],
    [32.77265929, 32.99170654, 32.6924274],
    [32.57145481, 32.78915724, 32.68150444],
    [32.43148648, 32.64825338, 32.67376734]
]

data7 = [
    [426.3351182, 439.3512247, 459.3200341],
    [142.1117061, 146.4504082, 139.327077],
    [66.31879616, 68.34352384, 64.30480477],
    [36.94904357, 38.07710614, 36.97526275],
    [36.94904357, 38.07710614, 37.51113612],
    [37.67353462, 38.82371606, 37.89390281]
]
```

```
data8 = [
    [463.2310365, 303.9317369],
    [145.5868972, 113.9744013],
    [69.48465547, 50.65528949],
    [40.00631679, 27.13676223],
    [40.97813015, 26.8175062],
    [41.69079328, 26.59402698]
]

# Calculer les moyennes pour chaque liste de données
means1 = calculate_means(data1)
means2 = calculate_means(data2)
means3 = calculate_means(data3)
means4 = calculate_means(data4)
means5 = calculate_means(data5)
means6 = calculate_means(data6)
means7 = calculate_means(data7)
means8 = calculate_means(data8)

# Tracer les courbes pour toutes les listes de données sur le même graphique
x = np.arange(6)
plt.plot(x, means1, marker='o', linestyle='-',color='saddlebrown' )
plt.plot(x, means2, marker='o', linestyle='-', color='lightblue')
plt.plot(x, means3, marker='o', linestyle='-', color='plum')
plt.plot(x, means4, marker='o', linestyle='-', color='darkorange')
plt.plot(x, means5, marker='o', linestyle='-',color='red' )
plt.plot(x, means6, marker='o', linestyle='-', color='gold')
plt.plot(x, means7, marker='o', linestyle='-', color='darkgreen')
plt.plot(x, means8, marker='o', linestyle='-',color='darkblue' )

plt.xlabel('nombre de constructions')
plt.ylabel('Temps mis pour rembourser son investissement')
plt.title('évolution du temps mis pour rembourser son investissement en fonction du nombre de constructions')
plt.legend()
plt.grid(True)
plt.show()
```

```python
import numpy as np
import matplotlib.pyplot as plt

m = max(max(l) for l in PInEXP_test)
print(m)
xlabels = [
    'Boulevard de...', 'Rue Lecourbe', 'Rue Vaugirard', 'Rue de Courcelles', 'Avenue de la...',
    'Boulevard de la...', 'Avenue de Neuilly', 'Rue de Paradis', 'Avenue Mozart',
    'Boulevard Saint...', 'Place Pigalle', 'Avenue Matignon', 'Boulevard...', 'Avenue Henri...',
    'Faubourg Saint...', 'Place de la Bourse',  'Rue Lafayette', 'Avenue de Breteuil',
    'Avenue Foch', 'Boulevard des...', 'Avenue des...', 'Rue de la Paix'
]
ylabels = [str(i) for i in range(m)]
data = np.zeros((m+1, 22))

for l in PInEXP_test:
    for j in range(22):
        data[l[j], j] += 1


fig, ax = plt.subplots()
heatmap = ax.pcolor(data)


# Show all ticks and label them with the respective list entries
ax.set_xticks(np.arange(len(xlabels)), labels=xlabels)
ax.set_yticks(np.arange(len(ylabels)), labels=ylabels)

# Rotate the tick labels and set their alignment.
plt.setp(ax.get_xticklabels(), rotation=90, ha="right", rotation_mode="anchor")

for j in range(21):
    ax.axhline(xmin=j/22, xmax=(j+1)/22, y=PINPROP3[j], color='red', linestyle='-', linewidth=2)

plt.colorbar(heatmap)
fig.tight_layout()
plt.show()
```

1) Propriétés de la matrice stochastique

Propriétés : pour tous $s, t \geq 0$,

a) $P(t)$ est une matrice stochastique : $p_{x,y}(t) \geq 0$ et $\sum_y p_{x,y}(t) = 1$ pour tout x ;

b) $P(t+s) = P(t)P(s)$;

c) $\vec{\pi}(t+s) = \vec{\pi}(t)P(s)$.

Cas discret (Monopoly) : la démonstration est identique avec $t, s \in \mathbb{N}$. On obtient $P(n+k) = P(n)P(k)$, d'où, par récurrence,

$$P(n) = P(1)^n \qquad \text{et} \qquad \vec{\pi}(n) = \vec{\pi}(0)\, P(1)^n.$$

Preuve : page suivante.

a) $p_{x,y}(t) \in [0,1]$ car c'est une probabilité. De plus, la ligne x correspond à la loi $P_{X_t}^{[X_0=x]}$ et on a bien

$$\sum_y p_{x,y}(t) = \sum_y P^{[X_0=x]}([X_t = y]) = 1.$$

b) Pour calculer $p_{x,y}(t+s) = P^{[X_0=x]}([X_{t+s} = y])$, on va faire intervenir les différents états pouvant être occupés à l'instant t :

$$\begin{aligned}
P^{[X_0=x]}([X_{t+s} = y]) &= \frac{P([X_0 = x] \cap [X_{t+s} = y])}{P([X_0 = x])} \\
&= \sum_{z \in E} \frac{P([X_0 = x] \cap [X_t = z] \cap [X_{t+s} = y])}{P([X_0 = x])} \\
&= \sum_{z \in E} \frac{P([X_{t+s} = y]/[X_0 = x] \cap [X_t = z]) P([X_0 = x] \cap [X_t = z])}{P([X_0 = x])} \\
&= \sum_{z \in E} P([X_{t+s} = y]/[X_t = z]) P([X_t = z]/[X_0 = x])
\end{aligned}$$

d'après l'axiome de Markov (X_0 n'apporte rien de plus que X_t pour déterminer la loi de X_{t+s}). Ainsi, on a

$$p_{x,y}(t+s) = \sum_{z \in E} p_{x,z}(t) p_{z,y}(s),$$

c) $\pi_y(t+s) = P([X_{t+s} = y]) = \sum_{x \in E} P([X_{t+s} = y]/[X_t = x]) P([X_t = x])$ c'est-à-dire

$$\pi_y(t+s) = \sum_{x \in E} \pi_x(t) p_{x,y}(s).$$

Cette équation différentielle matricielle admet l'unique solution :

$$\boxed{P(t) = e^{tA} = \sum_{n=0}^{+\infty} \frac{t^n}{n!} A^n.}$$

Ainsi, on détermine A à partir de la famille $(P(t))_{t\geq 0}$, $(A = P'(0))$, mais réciproquem seule connaissance de A permet de retrouver tous les $P(t)$ $(P(t) = e^{At})$.

La matrice A est appelée le *générateur infinitesimal* du processus.

On pose $A = (a_{x,y})_{x,y\in E}$. Ainsi, $a_{x,y} = \lim_{h\to 0} \dfrac{p_{x,y}(h) - \delta_{x,y}}{h}$.

- Si $x \neq y$, $a_{x,y} \geq 0$ et $p_{x,y}(h) = a_{x,y}h + o(h)$
- Si $x = y$, $a_{x,x} \leq 0$ et $p_{x,x}(h) = 1 + a_{x,x}h + o(h)$.

On a ici, $\boxed{\text{pour tout } x \in E, \sum_{y\in E} a_{x,y} = 0}$ (la somme sur chaque ligne de A vaut 0).

Propriété : Pour tout état $x \in E$, le temps passé dans x avant de le quitter suit la loi exponentielle $\mathcal{E}(\lambda_x)$ avec $\lambda_x = -a_{x,x} = \sum_{y \neq x} a_{x,y}$.

Preuve : Si T_x désigne ici le temps passé en x avant de le quitter, on a :

$$P([T_x > t+h]) = P([T_x > t+h]/[T_x > t])P([T_x > t])$$

Mais, d'après l'homogénéité dans le temps, on a $P([T_x > t+h]/[T_x > t]) = P([T_x > h])$. Ainsi, si on pose $G_x(t) = P([T_x > t])$, on obtient :

$$G_x(t+h) = G_x(t)G_x(h) \text{ avec } G_x(h) = p_{x,x}(h) + o(h) = 1 + a_{x,x}h + o(h),$$

donc $\dfrac{G_x(t+h) - G_x(t)}{h} = G_x(t)\left(a_{x,x} + \dfrac{o(h)}{h}\right)$ et, en faisant $h \to 0$, il vient :

$$G'_x(t) = a_{x,x}G_x(t) \text{ avec } G_x(0) = 1,$$

soit $G_x(t) = e^{a_{x,x}t}$ et $\boxed{P([T_x \leq t]) = 1 - e^{a_{x,x}t}}$.

□

L'évolution d'un processus de Markov à temps continu peut se voir comme une répétition de deux phases :

→ on reste un certain temps (de loi exponentielle) dans un état ;

→ lorsqu'on quitte cet état, on choisit l'état vers lequel on sort, cette destination ne dépendant ni du temps passé dans l'état, ni du chemin par lequel on est arrivé à l'état. On notera $p_{x,y}$ la probabilité de se rendre dans l'état y en quittant l'état x.

Propriété : Pour tout $(x,y) \in E^2$ tel que $x \neq y$, $p_{x,y} = \dfrac{a_{x,y}}{\sum_{z \neq x} a_{x,z}}$ (et $p_{x,x} = 0$).

Preuve : On a, si $\lambda_x = -a_{x,x} = \sum_{z \neq x} a_{x,z}$,

$$P([X_{t+h} \neq x]/[X_t = x]) = \lambda_x h + o(h)$$

et, pour $y \neq x$,

$$P([X_{t+h} = y]/[X_t = x]) = a_{x,y}h + o(h) = p_{x,y}\lambda_x h + o(h)$$

d'après la définition de $p_{x,y}$. C'est donc bien que $p_{x,y} = \dfrac{a_{x,y}}{\lambda_x} = \dfrac{a_{x,y}}{\sum\limits_{z \neq x} a_{x,z}}$.

Théorème : (Formule de Little) Le nombre moyen de clients, le temps moyen passé dans le système et le débit moyen d'un système stable en régime permanent se relient de la façon suivante :

$$L = W \times d$$

Ici $d = \lambda$ car $\lambda_n = \lambda$ pour tout $n \geq 0$. Une autre façon de voir les choses est de remarquer que le service s'effectue avec un taux μ dans chaque état où le système contient au moins un client :

$$d = \text{Proba}([\text{file non vide}])\mu = \sum_{n=1}^{\infty} \pi_n \mu = [1 - \pi_0]\mu = \rho\mu = \lambda$$

On retrouve bien que si la file est stable, le débit moyen de sortie est égal au débit moyen d'entrée.

On rappelle que, si $\overrightarrow{\pi}(t)$ est la loi de X_t, *i.e.* $\overrightarrow{\pi}(t) = (\pi_x(t))_{x\in E}$ où $\pi_x(t) = P([X_t = x])$, on a :

$$\overrightarrow{\pi}(t) = \overrightarrow{\pi}(0)P(t).$$

Définition : On dit que le processus $(X_t)_{t\geq 0}$ est *stationnaire* si la loi de X_t est indépendante de t.

On a alors $\overrightarrow{\pi}(t) = \overrightarrow{\pi}(0)$ pour tout $t \geq 0$ et en dérivant l'équation précédente, on obtient :

$$0 = \overrightarrow{\pi}(0)P'(t) = \overrightarrow{\pi}(0)P(t)A = \overrightarrow{\pi}(t)A = \overrightarrow{\pi}(0)A.$$

Définition : On appelle *distribution stationnaire* toute probabilité $\overrightarrow{\pi}$ qui vérifie

$$\overrightarrow{\pi}A = \overrightarrow{0}.$$

Si (X_t) converge en loi et si $\overrightarrow{\pi}^{(\infty)} = \lim_{t\to+\infty} \overrightarrow{\pi}(t)$, alors $\overrightarrow{\pi}^{(\infty)}$ est distribution stationnaire du processus.

On a $(\overrightarrow{\pi}A)_y = \sum_{x\in E} \pi_x a_{x,y} = \sum_{x\neq y} \pi_x a_{x,y} + \pi_y a_{y,y}$ avec $a_{y,y} = -\sum_{x\neq y} a_{y,x}$ ainsi :

$$\overrightarrow{\pi}A = \overrightarrow{0} \text{ équivaut à } \sum_{x\neq y} \pi_x a_{x,y} = \sum_{x\neq y} \pi_y a_{y,x}$$

Cette relation traduit l'équilibre, en régime stationnaire du flux rentrant en y $(\sum_{x\neq y} \pi_x a_{x,y})$ et du flux sortant de y $(\sum_{x\neq y} \pi_y a_{y,x})$.

```python
## file M/M/1

import numpy as np
import random
import matplotlib.pyplot as plt
import math


# Paramètres de la file d'attente M/M/1
arrival_rate = 0.2  # Taux moyen d'arrivée
service_rate = 0.3  # Taux moyen de service
simulation_time = 36000  # Temps total de simulation

def generate_interarrival_time(arrival_rate):
    return np.random.exponential(1/arrival_rate)

def generate_service_time(service_rate):
    return np.random.exponential(1/service_rate)

def mm1_queue(arrival_rate, service_rate, simulation_time):
    queue = []
    clock = 0.0
    total_wait_time = 0.0
    num_served = 0

    while clock < simulation_time:
        if len(queue) == 0:
            arrival_time = clock + generate_interarrival_time(arrival_rate)
            service_time = generate_service_time(service_rate)
            departure_time = arrival_time + service_time
            queue.append(departure_time)
        else:
            queue.sort()
            arrival_time = clock + generate_interarrival_time(arrival_rate)
            if arrival_time < queue[0]:
                wait_time = queue[0] - arrival_time
                total_wait_time += wait_time
                num_served += 1
                service_time = generate_service_time(service_rate)
                departure_time = queue[0] + service_time
                queue[0] = departure_time
            else:
                arrival_time = queue[0] + generate_interarrival_time(arrival_rate)
                service_time = generate_service_time(service_rate)
                departure_time = arrival_time + service_time
                queue.append(departure_time)
                del queue[0]
        clock = arrival_time

    if num_served >= 1 :
        average_wait_time = total_wait_time / num_served
    else:
```

```
    if num_served >= 1 :
        average_wait_time = total_wait_time / num_served
    else:
        average_wait_time=100
    return average_wait_time


# Simulation

average_wait_time = mm1_queue(arrival_rate ,service_rate, simulation_time)
print("Temps d'attente moyen :", average_wait_time)

# Paramètres du graphe
num_values = 1000  # Nombre de valeurs à générer
x_values = range(num_values)

y_values = [mm1_queue(arrival_rate, service_rate, simulation_time) for _ in x_values]  # Valeurs y
z_value=np.mean([mm1_queue(arrival_rate, service_rate, simulation_time) for _ in
x_values])*np.ones(num_values)
print(z_value)

# Affichage du graphe


plt.scatter(y_values, x_values)
plt.scatter(z_value,x_values)

plt.xlabel("temps moyen passé dans la file")
plt.ylabel("numéro d'essai")
plt.title("File d'attente M/M/1 : 1000 files d'attentes pendant 10 heures" )
plt.text(0.8,0.8,z_value[0],transform=plt.gca().transAxes)
plt.show()
```

```python
## affichage de fonctions de deux variables
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

#paramètre de la file d'attente
num_points= 50  #nombre de points à tracer
arrival_rates=np.linspace(0.01,0.99,num_points) #taux d'arrivée
service_rates=np.linspace(0.02,1.00,num_points) #taux de service

wait_times=np.zeros((num_points,num_points))
for i in range(num_points):
    for j in range (num_points):
        wait_times[i][j]= mm1_queue(arrival_rates[i],service_rates[j],3600)

X, Y = np.meshgrid(arrival_rates, service_rates) #mieux cerner la fonction

# Création de la figure
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

# Affichage de la fonction
ax.plot_surface(X, Y, wait_times)

# Étiquettes des axes
ax.set_xlabel('taux darrivée')
ax.set_ylabel('taux de service')
ax.set_zlabel('temps d_attente moyen')

# Affichage de la figure
plt.show()
```

```
##Simulation faites pour les 2 files d'attentes M/M/1
# Simulation pour la file d'attente 1
num_served_1, total_wait_time1= mm1_queue(ARRIVAL_RATE_1, SERVICE_RATE_1, SIMULATION_TIME)

# Simulation pour la file d'attente 2
num_served_2, total_wait_time2 = mm1_queue(ARRIVAL_RATE_2, SERVICE_RATE_2, SIMULATION_TIME)

average_wait_time_total2=(total_wait_time1+total_wait_time2)/(num_served_1 + num_served_2)
print("Le temps d'attente moyen est : ", average_wait_time_total2)
# Définir le nombre de répétitions
num_repetitions = 1000

# Liste pour stocker les valeurs de average_wait_time_total2
average_wait_times = []

# Effectuer les répétitions
for _ in range(num_repetitions):
    # Effectuer la simulation pour la file d'attente 1
    num_served_1, total_wait_time1 = mm1_queue(ARRIVAL_RATE_1, SERVICE_RATE_1, SIMULATION_TIME)

    # Effectuer la simulation pour la file d'attente 2
    num_served_2, total_wait_time2 = mm1_queue(ARRIVAL_RATE_2, SERVICE_RATE_2, SIMULATION_TIME)

    # Calculer le temps d'attente moyen total
    average_wait_time_total2 = (total_wait_time1 + total_wait_time2) / (num_served_1 + num_served_2)

    # Ajouter la valeur à la liste
    average_wait_times.append(average_wait_time_total2)
# Calcul de la population moyenne en fonction du temps
population = []
time = []



# Tracer le graphe
plt.plot(time, population)
plt.xlabel('Temps')
plt.ylabel('Population moyenne')
plt.title('Evolution de la population moyenne en fonction du temps')
plt.grid(True)
plt.show()
```

```python
## FILE M/M/2 avec affichage de la population + genre
import random
import matplotlib.pyplot as plt
import numpy as np

# Paramètres de la file d'attente M/M/2
ARRIVAL_RATE = 0.4
SERVICE_RATE_MEN = 0.3
SERVICE_RATE_WOMEN = 0.3
SIMULATION_TIME = 36000

# Listes pour stocker les données
population_men = []
population_women = []
population_total = []
time = []
queue = []

def mm2_queue(arrival_rate, service_rate_men, service_rate_women, simulation_time):
    clock = 0.0
    total_wait_time_men = 0.0
    total_wait_time_women = 0.0
    num_served_men = 0
    num_served_women = 0

    server_men_busy_until = 0.0
    server_women_busy_until = 0.0

    population_total = []  # Liste pour stocker le nombre total de clients servis dans chaque simulation

    while clock < simulation_time:
        interarrival_time = random.expovariate(arrival_rate)
        clock += interarrival_time

        gender = random.choices(['M', 'W'], weights=[0.8, 0.2])[0]
        service_time = random.expovariate(service_rate_men if gender == 'M' else service_rate_women)
        queue.append([clock, 1])

        if gender == 'M':
            wait_time = max(0, server_men_busy_until - clock)
            total_wait_time_men += wait_time
            num_served_men += 1
            server_men_busy_until = clock + service_time
            queue.append([server_men_busy_until, -1])
        else:
            wait_time = max(0, server_women_busy_until - clock)
            total_wait_time_women += wait_time
            num_served_women += 1
            server_women_busy_until = clock + service_time
            queue.append([server_women_busy_until, -1])
```

```
        population_total.append(num_served_men + num_served_women)  # Ajouter le nombre total de clients
servis dans la simulation courante

        time.append(clock)

    average_wait_time_men = total_wait_time_men / num_served_men
    average_wait_time_women = total_wait_time_women / num_served_women
    average_wait_time_total = (total_wait_time_men + total_wait_time_women) / (num_served_men +
num_served_women)
    return population_total[-1]  #la population_total contenant le nombre total de clients servis dans chaque
simulation

# Simulation
population_total = mm2_queue(ARRIVAL_RATE, SERVICE_RATE_MEN, SERVICE_RATE_WOMEN, SIMULATION_TIME)


# Tracer le graphe
num_repetitions = 1000
x_values = list(range(num_repetitions))
y_values = []

for i in range(num_repetitions):
    y_values.append(mm2_queue(ARRIVAL_RATE, SERVICE_RATE_MEN, SERVICE_RATE_WOMEN, SIMULATION_TIME))

z_value = np.mean(y_values)

plt.scatter(x_values, y_values, label='Simulation')
plt.axhline(y=z_value, color='r', linestyle='--', label='Valeur moyenne')
plt.xlabel('Simulation')
plt.ylabel('Nombre de clients servis')
plt.title("Nombre moyen de clients servis sur 100 simulations de 10 heures")
plt.legend()
plt.show()
```